

Unit 1

DSBDA

Dr.Sangeeta Gupta

Associate Professor, CSE Dept, CBIT,
Hyderabad

MODELLING

- Rachel had a recent phone conversation with someone about a modeling workshop, and several minutes into it she realized the word “model” meant completely different things to them.
- He was using it to mean data models—the representation one is choosing to store one’s data, related to database managers.
- She was talking about talking about statistical models.
- What’s the difference between a statistical model and a machine learning algorithm?

What is a model?

- Humans try to understand the world around them by representing it in different ways.
- Architects capture attributes of buildings through blueprints and three-dimensional, scaled-down versions.
- Statisticians and data scientists capture the uncertainty and randomness of data-generating processes with mathematical functions that express the shape and structure of the data itself.
- ***A model is an artificial construction where all extraneous detail has been removed or abstracted. Attention must always be paid to these abstracted details after a model has been analyzed to see what might have been overlooked.***

Statistical modeling

- Before dealing with data and start coding
 - underlying process of model.
 - What comes first?
 - What influences what?
 - What causes what?
 - What's a test of that?
- In mathematical expressions, Greek letters for parameters and Latin letters for data is used.
- Ex:-if you have two columns of data, x and y , and you think there's a linear relationship, you'd write down $y = \beta_0 + \beta_1 x$.
- You don't know what β_0 and β_1 are in terms of actual numbers yet, so they're the parameters.

But how do you build a model?

- What functional form the data should take?
- lot of assumptions about the underlying structure of reality.
- Start with exploratory data analysis (EDA)
- EDA helps out a lot, as well as trial and error and iteration.
- Some of the building blocks of this model are probability distribution.

Probability distribution

- The foundation of statistical models.
- Linear regression and Naive Bayes.
- EX:- before computers, scientists observed real-world phenomena, took measurements, and noticed that certain mathematical shapes kept reappearing.
- The classical example is the height of humans, following a normal distribution—a bell-shaped curve, also called a Gaussian distribution, named after Gauss.
- Other common shapes have been named after their observers as well (e.g., the Poisson distribution and the Weibull distribution), while other shapes such as Gamma distributions or exponential distributions are named after associated mathematical objects.
- Natural processes tend to generate measurements whose empirical shape could be approximated by mathematical functions with a few parameters that could be estimated from the data.
- Not all processes generate data that looks like a named distribution, but many do.

Probability distribution

$$N(x|\mu, \sigma) \sim \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

The parameter μ is the mean and median and controls where the distribution is centered (because this is a symmetric distribution), and the parameter σ controls how spread out the distribution is. This is the general functional form, but for specific real-world phenomenon, these parameters have actual numbers as values, which we can estimate from the data.

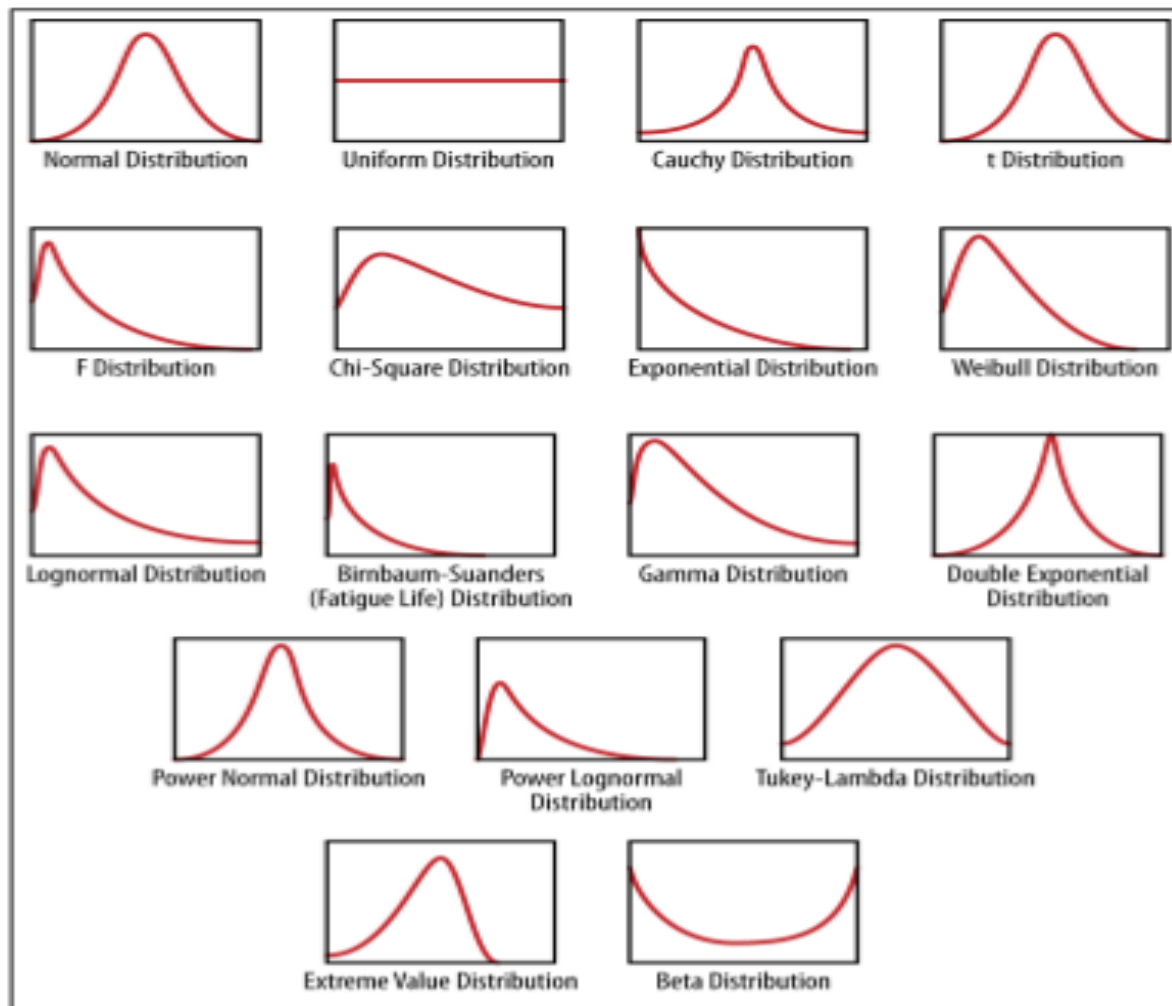


Figure 2-1. A bunch of continuous density functions (aka probability distributions)

Probability distribution

- A random variable denoted by x or y can be assumed to have a corresponding probability distribution $p(x)$, which maps x to a positive real number.
- if we integrate $p(x)$ to get the area under the curve, it is 1, so it can be interpreted as probability.
- Ex:- let x be the amount of time until the next bus arrives (measured in minutes). x is a random variable because there is variation and uncertainty in the amount of time until the next bus.
- Suppose we know (for the sake of argument) that the time until the next bus has a probability density function of $p(x) = 2e^{-2x}$.
- If we want to know the likelihood of the next bus arriving in between 12 and 13 minutes, then we find the area under the curve between 12 and 13 by

$$\int_{12}^{13} 2e^{-2x}.$$

- How do we know this is the right distribution to use?
- Wait time is common enough in real world phenomenon that a distribution is called as ***Exponential Distribution***. $p(x) = \lambda e^{-\lambda x}$.
- When denoting single random variables with functions of one variable, we use multivariate functions called ***joint distributions*** to do the same thing for more than one random variable.
- We also have what is called a ***conditional distribution***, $p(x|y)$, which is to be interpreted as the density function of x given a particular value of y .

Fitting a model

- you estimate the parameters of the model using the observed data.
- Fitting the model often involves optimization methods and algorithms, such as maximum likelihood estimation, to help get the parameters.
- when you estimate the parameters, they are actually estimators, which means they themselves are functions of the data.
- Once you fit the model, you actually can write it as $y = 7.2 + 4.5x$, (expresses the relationship between your two variables, based on your assumption that the data followed a linear pattern.)

Fitting a model

- Fitting the model is when you start actually coding:
 - your code will read in the data,
 - You will specify the functional form that you wrote down
 - Then R or Python will use built-in optimization methods .
- Overfitting
 - Overfitting is the term used to mean that you used a dataset to estimate the parameters of your model, but your model isn't that good at capturing reality beyond your sampled data.

EDA(EXPLORATORY DATA ANALYSIS)

- “Exploratory data analysis” is an attitude, a state of flexibility, a willingness to look for those things that we believe are not there, as well as those we believe to be there.
— John Tukey
- Bell Labs in 1920s that has made innovations in physics, computer science, statistics, and math, producing languages like C++, and many Nobel Prize winners as well.
- There was a very successful and productive statistics group there, and among its many notable members was John Tukey, a mathematician who worked on a lot of statistical problems.
- He is considered the father of EDA and R (which started as the S language at Bell Labs; R is the open source version), and he was interested in trying to visualize high-dimensional data.

EDA

- EDA is a critical part of the data science process, and also represents a philosophy or way of doing statistics
- The basic tools of EDA are plots, graphs and summary statistics.
- Computing their mean, minimum, maximum, the upper and lower quartiles, and identifying outliers.
- EDA is a set of tools, it's also a mindset.
- And that mindset is about your relationship with the data.
 - You understand the data—gain intuition,
 - understand the shape of it,
 - try to connect your understanding of the process

Philosophy of Exploratory Data Analysis

- Even with very large Google-scale data, they did EDA. In the context of data in an Internet/engineering company, EDA is done for some of the same reasons it's done with smaller datasets.
- There are important reasons anyone working with data should do EDA
 - to gain intuition about the data;
 - to make comparisons between distributions;
 - for sanity checking (making sure the data is on the scale you expect, in the format you thought it should be);
 - to find out where data is missing or if there are outliers;
 - and to summarize the data.
- EDA also helps with debugging the logging process.
- If you never go to the trouble of debugging, you'll continue to think your patterns are real.

Exercise: EDA

- There are 31 datasets named nyt1.csv, nyt2.csv,...,nyt31.csv, which you can find here: https://github.com/oreillymedia/doing_data_science.
- Each one represents one (simulated) day's worth of ads shown and clicks recorded on the New York Times home page in May 2012.
- . Each row represents a single user.
- There are five columns: age, gender (0=female, 1=male), number impressions, number clicks, and loggedin.

You'll be using R to handle these data

- `data1 <- read.csv(url("http://stat.columbia.edu/~rachel/datasets/nyt1.csv"))`
- Once you have the data loaded, it's time for some EDA:
 - 1. Create a new variable, `age_group`, that categorizes users as "<18", "18-24", "25-34", "35-44", "45-54", "55-64", and "65+".
 - 2. For a single day:
 - Plot the distributions of number impressions and clickthrough-rate (CTR=# clicks/# impressions) for these six age categories.
 - Define a new variable to segment or categorize users based on their click behavior.
 - Explore the data and make visual and quantitative comparisons across user segments/demographics (<18-year-old males versus < 18-year-old females or logged-in versus not, for example).
 - Create metrics/measurements/statistics that summarize the data.
 - Examples of potential metrics include CTR, quantiles, mean, median, variance, and max, and these can be calculated across the various user segments. Be selective. Think about what will be important to track over time—what will compress the data, but still capture user behavior.
 - 3. Now extend your analysis across days. Visualize some metrics and distributions over time.
 - 4. Describe and interpret any patterns you find.

```
# Author: Maura Fitzgerald
```

```
data1 <- read.csv(url("http://stat.columbia.edu/~rachel/  
                      datasets/nyt1.csv"))
```

```
# categorize
```

```
head(data1)
```

```
data1$agecat <- cut(data1$Age, c(-Inf, 0, 18, 24, 34, 44, 54, 64, Inf))
```

```
# view
```

```
summary(data1)
```

```
# brackets
```

```
install.packages("doBy")
```

```
library("doBy")
```

```
siterange <- function(x){c(length(x), min(x), mean(x), max(x))}
```

```
summaryBy(Age~agecat, data =data1, FUN=siterange)
```

```
# so only signed in users have ages and genders
```

```
summaryBy(Gender+Signed_In+Impressions+Clicks~agecat,  
          data =data1)
```

```
# plot
```

```
install.packages("ggplot2")
```

```
library(ggplot2)
```

```
ggplot(data1, aes(x=Impressions, fill=agecat))
```

```
  +geom_histogram(binwidth=1)
```

```
ggplot(data1, aes(x=agecat, y=Impressions, fill=agecat))
```

```
  +geom_boxplot()
```

```

# create click thru rate
# we don't care about clicks if there are no impressions
# if there are clicks with noimps my assumptions about
# this data are wrong
data1$hasimps <- cut(data1$Impressions, c(-Inf, 0, Inf))
summaryBy(Clicks~hasimps, data = data1, FUN=siterange)
ggplot(subset(data1, Impressions>0), aes(x=Clicks/Impressions,
      colour=agecat)) + geom_density()
ggplot(subset(data1, Clicks>0), aes(x=Clicks/Impressions,
      colour=agecat)) + geom_density()
ggplot(subset(data1, Clicks>0), aes(x=agecat, y=Clicks,
      fill=agecat)) + geom_boxplot()
ggplot(subset(data1, Clicks>0), aes(x=Clicks, colour=agecat))
+ geom_density()

```

```

# create categories
data1$scode[data1$Impressions==0] <- "NoImps"
data1$scode[data1$Impressions >0] <- "Imps"
data1$scode[data1$Clicks >0] <- "Clicks"

```

```

# Convert the column to a factor
data1$scode <- factor(data1$scode)
head(data1)

```

```

#look at levels
clen <- function(x){c(length(x))}
etable<-summaryBy(Impressions~scode+Gender+agecat,
      data = data1, FUN=clen)

```

The Data Science Process

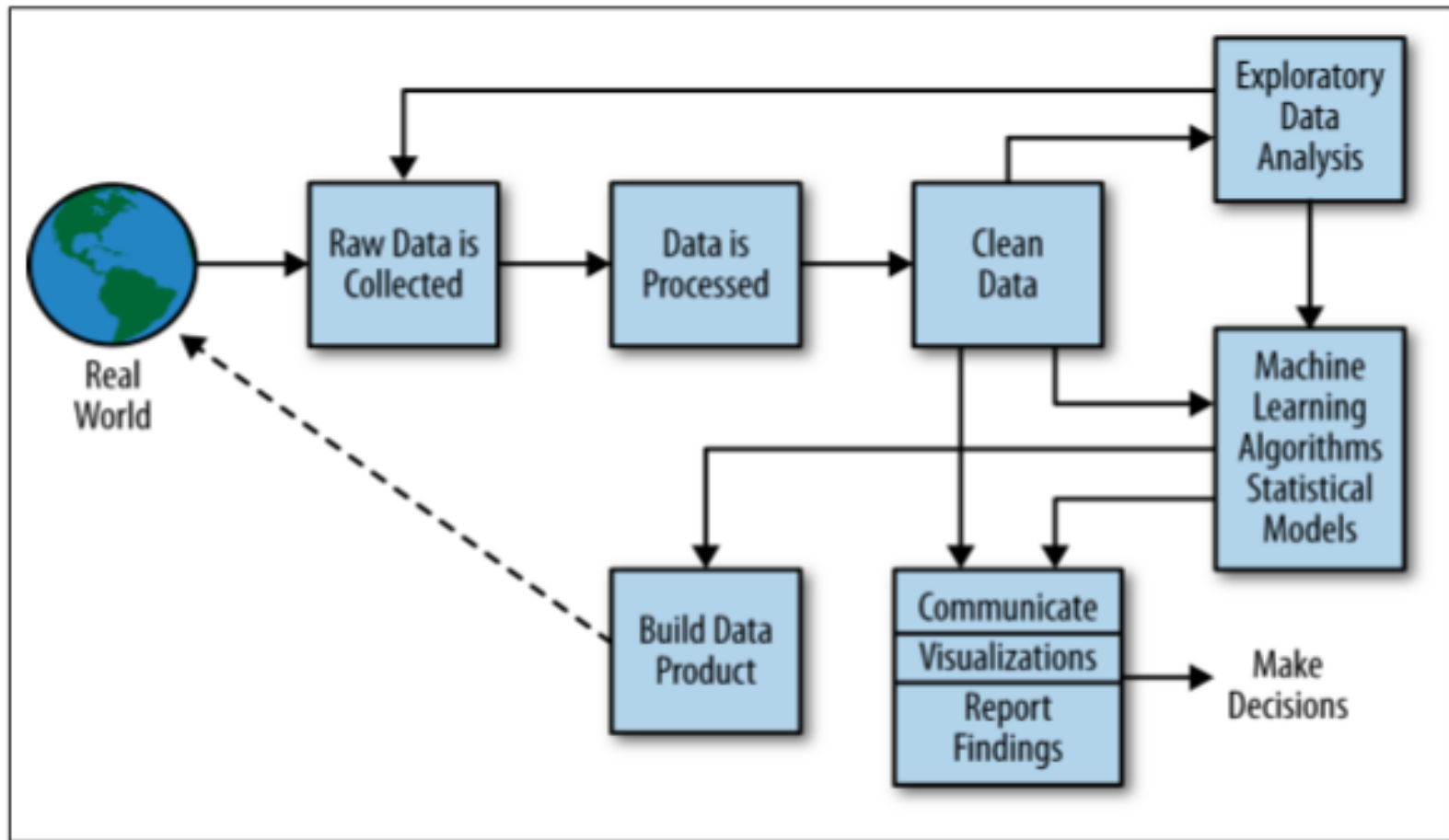


Figure 2-2. The data science process

The Data Science Process

- Start with raw data—logs, Olympics records, Enron employee emails, or recorded genetic material.
- We want to process this to make it clean for analysis
- we build and use pipelines of data munging: joining, scraping, wrangling,
- To do this we use tools such as Python, shell scripts, R, or SQL, or all of the above.
- Eventually we get the data down to a nice format, like something with columns:
 - name | event | year | gender | event time
- Once we have this clean dataset, we should be doing some kind of EDA.

- In the course of doing EDA, we may realize that it isn't actually clean because of duplicates, missing values, absurd outliers, and data that wasn't actually logged or incorrectly logged.
- If that's the case, we may have to go back to collect more data/ spend more time cleaning the dataset.
- We design our model to use some algorithm like k-nearest neighbor (k-NN), linear regression, Naive Bayes, or something else.
- Then can interpret, visualize, report, or communicate our results.
- Our goal may be to build or prototype a "data product"; e.g., a spam classifier, or a search ranking algorithm, or a recommendation system.
- Data product then gets incorporated back into the real world, and users interact with that product, and that generates more data, which creates a feedback loop.

A Data Scientist's Role in This Process

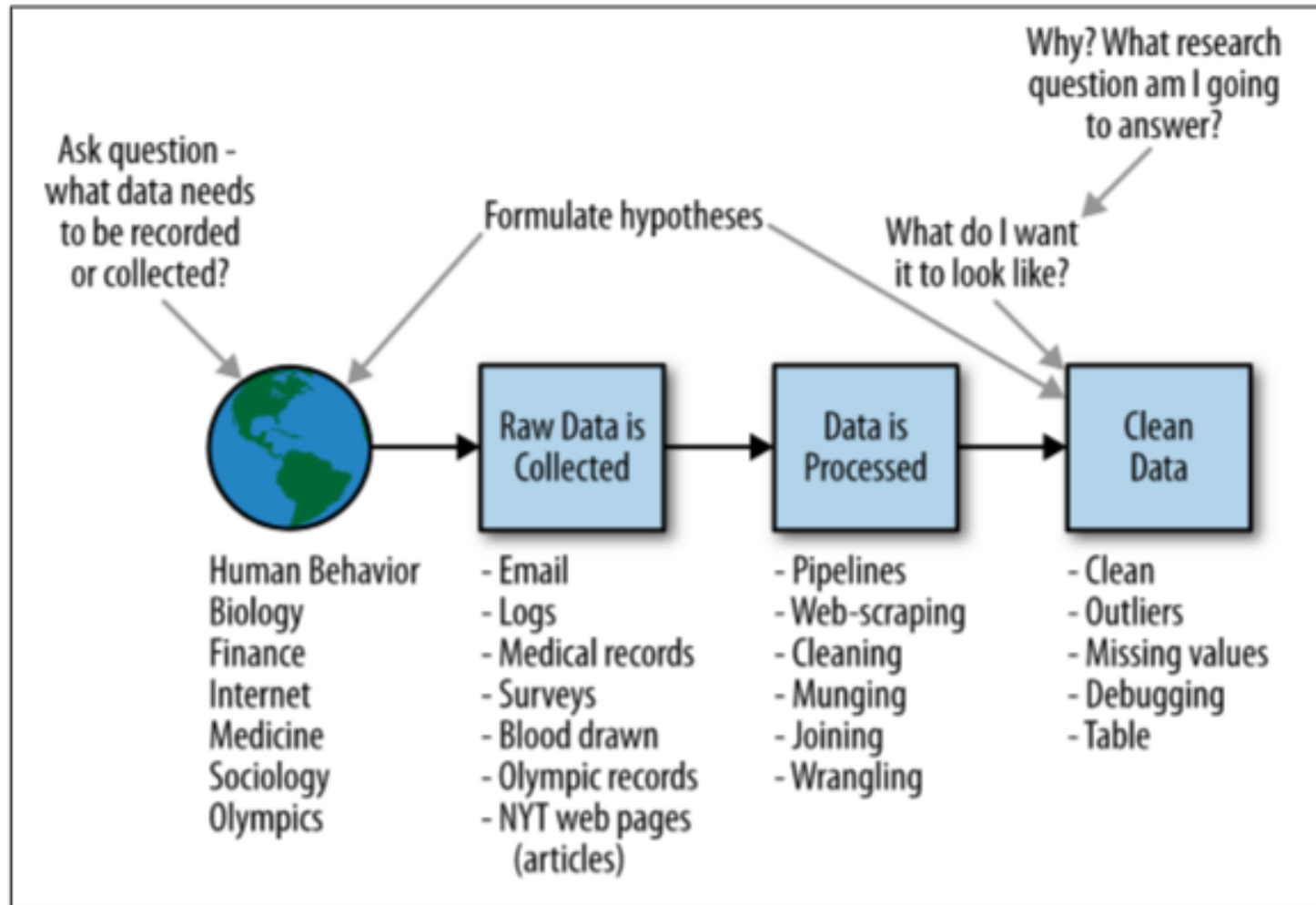


Figure 2-3. The data scientist is involved in every part of this process

A Data Scientist's Role in This Process

- Data Scientist has to make the decisions about what data to collect, and why.
- He/She needs to be formulating questions and hypotheses and making a plan for how the problem will be attacked.
- We can think of the data science process as an extension or variation of the scientific method:
 - Ask a question.
 - Do background research.
 - Construct a hypothesis.
 - Test your hypothesis by doing an experiment.
 - Analyze your data and draw a conclusion.
 - Communicate your results.

Discussion

- Big data and datascience hype
- Statistical Inference (Modelling, Probability Distribution)
- EDA
- The Data Science Process

UNIT 2

DSBDA

Dr.Sangeeta Gupta

Associate Professor, CSE Dept, CBIT,
Hyderabad

Machine Learning Algorithms

- These are largely used to predict , classify or cluster.(Ex:-Image Recognition).
- Data scientist should use both methods of machine learning and statisticians.
 - Interpreting Parameters
 - Confidence Interval
 - The role of Explicit assumption

- **Interpreting Parameters**
- Statisticians think in linear regression model.
- They want to find meaning ,behavior or describe the real world phenomenon corresponding to the parameters.
- Where as in S/W Engineering ,they want to build their linear regression algorithm into production level code or predictive model known as Black –box algorithm.
- No focus on interpretation of parameters

- **Confidence Intervals**
- Statisticians provide confidence intervals and posterior distribution for parameters and estimators.
- They are interested in capturing the variability and uncertainty of parameters .
- Machine learning algorithms like k-NN, k mean do not have confidence interval or uncertainty.

- **The role of Explicit Assumptions**
- Statisticians spent their life time on investigating uncertainty and are never 100% confident about anything .
- But s/w engineers build the things on assumptions
- They want to build the model or predictions
- Ex-Companies like FB , Google etc..

THREE BASIC ALGORITHMS

Linear Regression

- Common statistical method .
- Used to express the mathematical relation between 2 variables or attributes .
- Using it makes the assumption a linear relationship between an outcome variable and the predictor .
- between one variable and several other variables ,in such case your ***modeling*** the relationship as having a linear structure.
- Why would you want to build a linear model
- You may want to predict future outcomes or might want to understand or describe the relationships the get the situation .

- We can study 2 cases the relation ship between a company's sales and how much that company spends on advertising .
- Or the number of friends someone has on a social networking site and the time that person spends on that site daily
- Linear regression can be described as a line with slope

$$y = f(x) = \beta_0 + \beta_1 * x$$

Example 1(simplistic example)

- Suppose you run a social n/w site that charges a monthly subscription fee of \$25,this is you only source of revenue
- Each month you collect data and count your number of users and total revenue.
- You've done this daily over the course of two years, recording it all in a spreadsheet.
- Here are the first four:
- $S = (x,y) = \{(1,25),(10,250),(100,2500),(200,5000)\}$
- Clear relationship enjoyed by all of these points, namely $y=25x$.
 - There's a linear pattern.
 - The coefficient relating x and y is 25.
 - It seems deterministic.

Obvious Linear Pattern

You can even plot it as in **Figure 3-1** to verify they were right (even though you knew they were because you made the business model in the first place). It's a line!

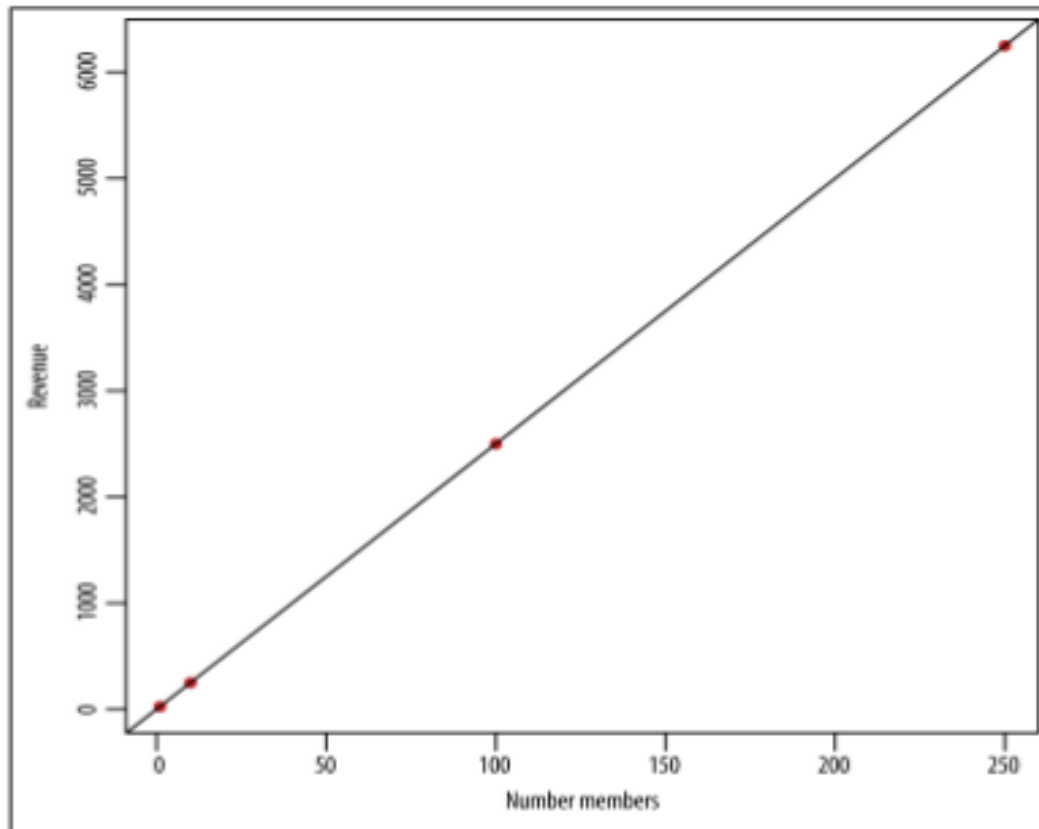


Figure 3-1. An obvious linear pattern

Example 2(Looking at data at user level)

- Consider a dataset keyed by user (meaning each row contains data for a single user), and the columns represent user behavior on a social networking site over a period of a week.
- Let's say data is clean at this stage and you have on the order of hundreds of thousands of users.
- The names of the columns are total_num_friends, total_new_friends_this_week, num_visits, time_spent, number_apps_downloaded, number_ads_shown, gender, age, and so on.
- During EDA ,you've randomly sampled 100 users to keep it simple, and you plot pairs of these variables,
 - EX:-x = total_new_friends and y = time_spent (in seconds).

- In business context you may want to be able to promise advertisers who bid for space on your website in advance a certain number of users, so you want to be able to forecast number of users several days or weeks in advance.
- But for now, you are simply trying to understand your dataset.
- You eyeball the first few rows and see:

7	276
3	43
4	82
6	136
10	417
9	269

- It is a linear relationship here, and it makes sense; the more new friends you have, the more time you might spend on the site

Now, your brain can't figure out what's going on by just looking at them (and your friend's brain probably can't, either). They're in no obvious particular order, and there are a lot of them. So you try to plot it as in **Figure 3-2**.

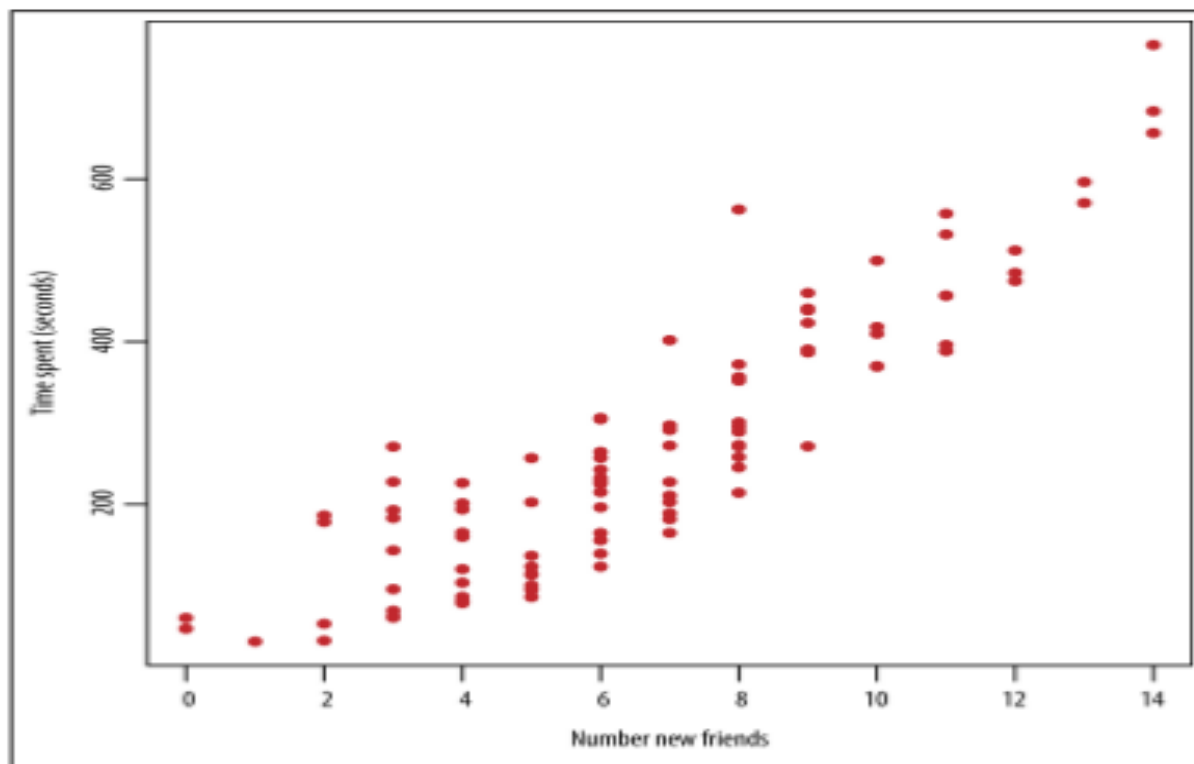


Figure 3-2. Looking kind of linear

- 2 things are captured in this model trend and variation.
- Starting with trend, considering there is actual relationship.
- assuming a linear relationship,
start your model by assuming the
functional form to be:

$$y = \beta_0 + \beta_1 x.$$

- Now your job is to find the best choices
for β_0 and β_1 using the observed data to
Estimate them: (x_1, y_1) , (x_2, y_2) ,... (x_n, y_n) .
Writing this with matrix notation results in this:
 $y = x \cdot \beta$
- There you go: you've written down your model.
Now the rest is fitting the model.

There are many lines that look more or less like they might work, as shown in **Figure 3-3**.

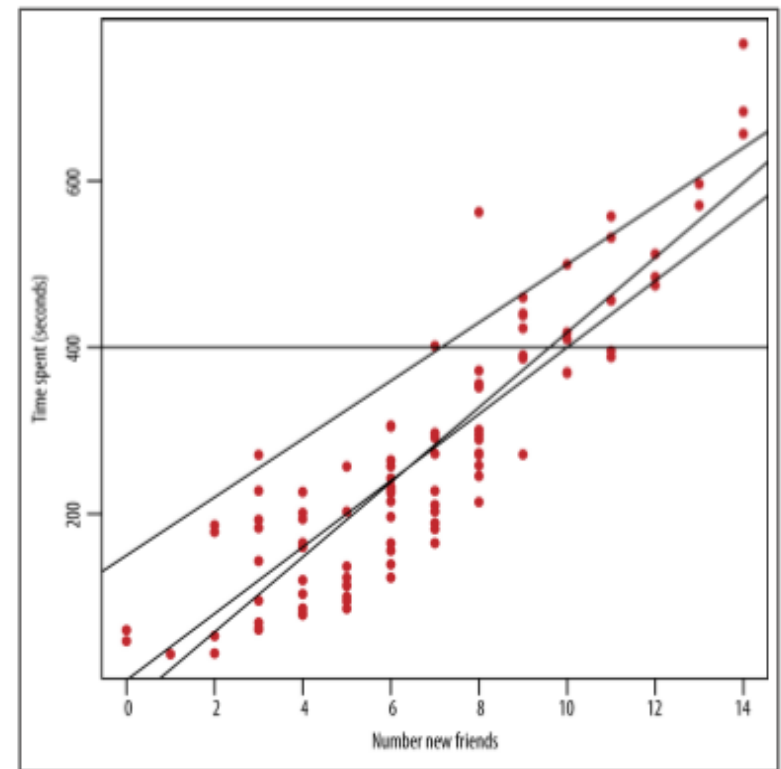


Figure 3-3. Which line is the best fit?

Fitting the model

- So, how do you calculate β ?
- The intuition behind linear regression is that you want to find the line that minimizes the distance between all the points and the line.
- Many lines look approximately correct, but your goal is to find the optimal one.
- Optimal could mean different things, optimal to mean the line that, on average, is closest to all the points.
- But what does closest mean here?

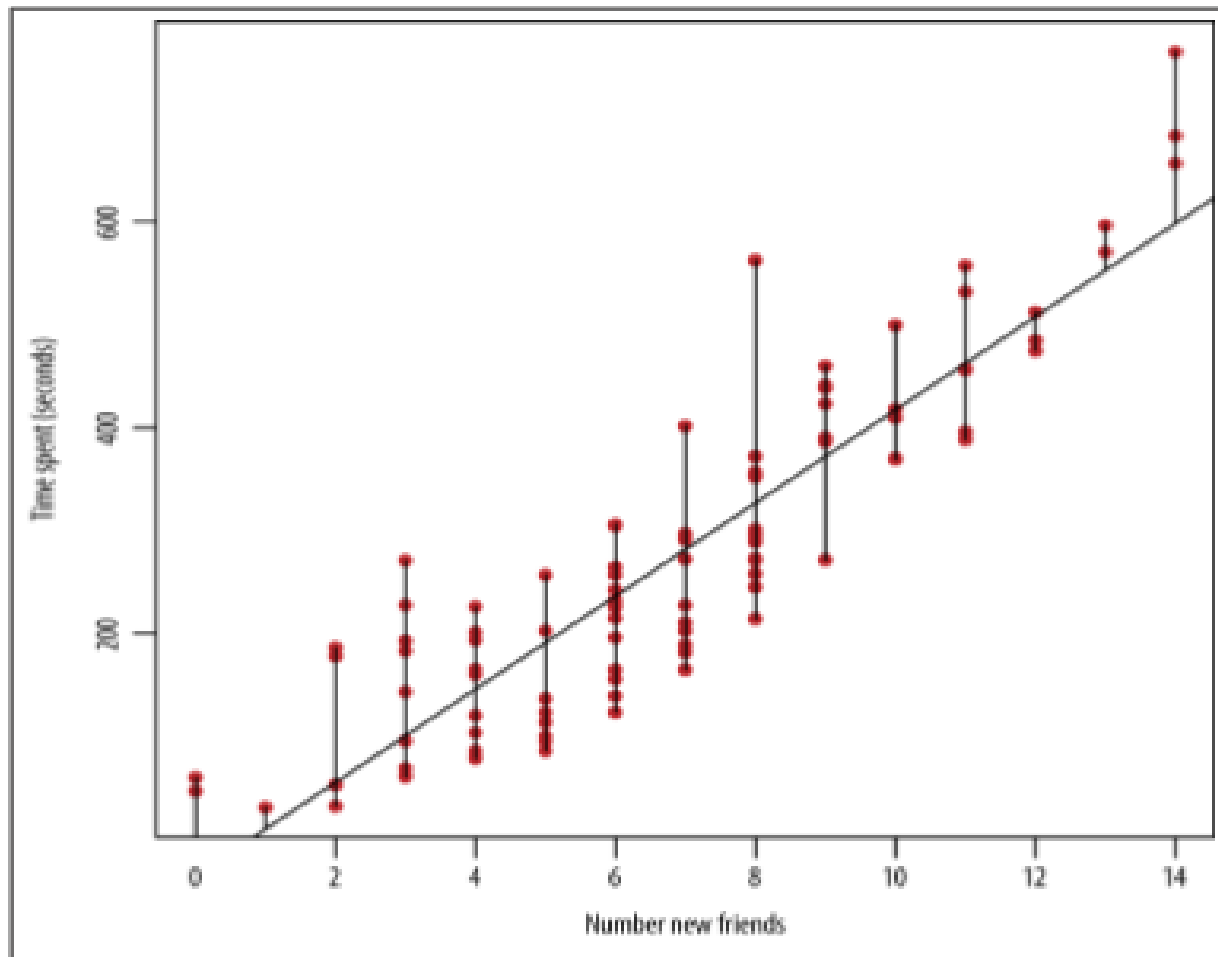


Figure 3-4. The line closest to all the points

To find this line, you'll define the "residual sum of squares" (RSS), denoted $RSS(\beta)$, to be:

$$RSS(\beta) = \sum_i (y_i - \beta x_i)^2$$

- Look at Figure 3-4.
- Linear regression seeks to find the line that minimizes the sum of the squares of the vertical distances between the approximated or predicted $\hat{y}_i$ s and the observed y_i s.
- You do this because you want to minimize your prediction errors.
- **This method is called least squares estimation.**
- To find this line, you'll define the “residual sum of squares” (RSS), denoted $RSS(\beta)$, to be:

$$RSS(\beta) = \sum_i (y_i - \beta x_i)^2$$

- where i ranges over the various data points.
- It is the sum of all the squared vertical distances between the observed points and any given line.
- Note this is a function of β and you want to optimize with respect to β to find the optimal line.

To minimize $RSS(\beta) = (y - \beta x)^t (y - \beta x)$, differentiate it with respect to β and set it equal to zero, then solve for β . This results in:

$$\hat{\beta} = (x^t x)^{-1} x^t y$$

- Here the little “hat” symbol on top of the β is there to indicate that it’s the estimator for β .
- Not true value of β ; all you have is the observed data, which you plug into the estimator to get an estimate.

- To actually fit this, to get the β s, all you need is one line of R code where you've got a column of y's and a (single) column of x's:

```
model <- lm(y ~ x)
```

So for the example where the first few rows of the data were:

x	y
7	276
3	43
4	82
6	136
10	417
9	269

The R code for this would be:

```
> model <- lm (y~x)
> model
```

```
Call:
lm(formula = y ~ x)
```

```
Coefficients:
(Intercept)          x
    -32.08         45.92
```

```
> coefs <- coef(model)
> plot(x, y, pch=20,col="red", xlab="Number new friends",
      ylab="Time spent (seconds)")
> abline(coefs[1],coefs[2])
```

And the estimated line is $\hat{y} = -32.08 + 45.92x$, which you're welcome to round to $\hat{y} = -32 + 46x$, and the corresponding plot looks like the lefthand side of **Figure 3-5**.

- But its up to u the data scientist if u want to use linear model or to describe the relationship or to predict the new outcome.
- So while you've so far modeled the trend, you haven't yet modeled the variation

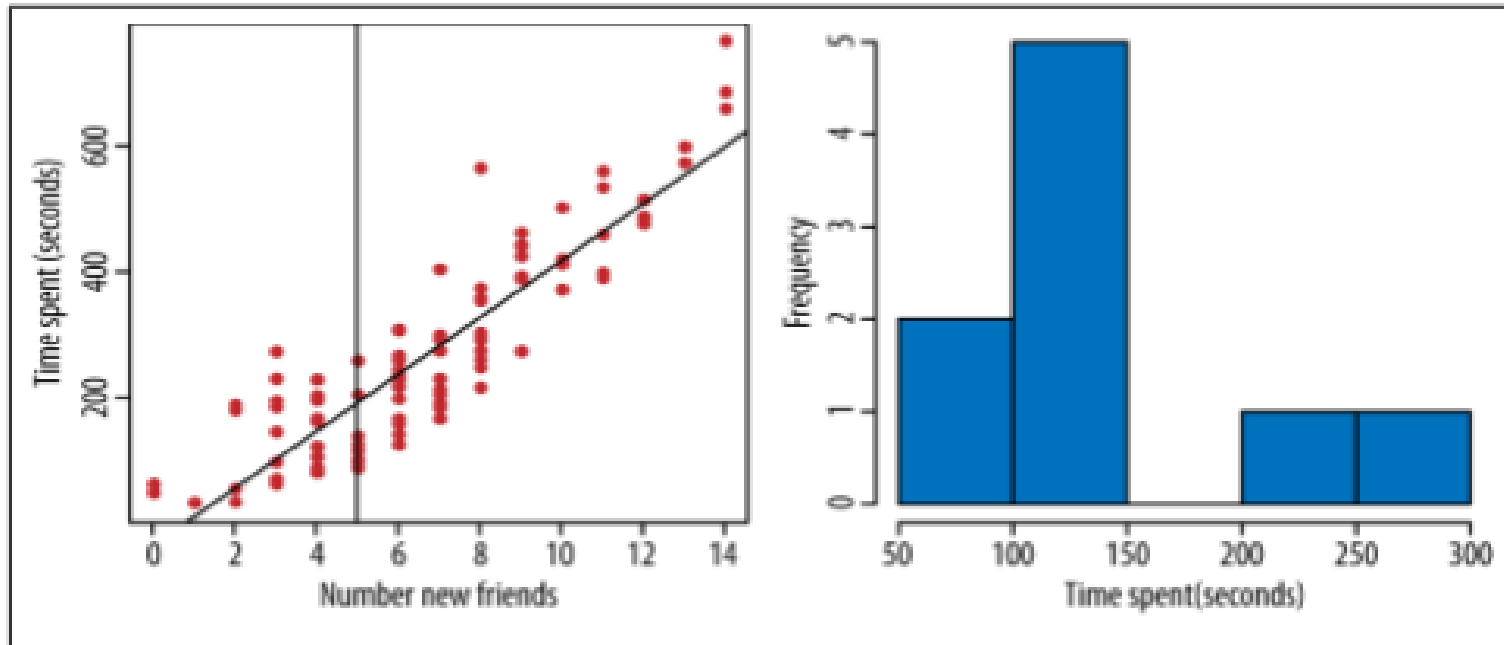


Figure 3-5. On the left is the fitted line. We can see that for any fixed value, say 5, the values for y vary. For people with 5 new friends, we display their time spent in the plot on the right.

Extending beyond least square

- The ***least squares method*** is a statistical procedure to find the best fit for a set of data points by minimizing the sum of the offsets or residuals of points from the plotted curve.
- Now that you have a simple linear regression model down (one output, one predictor) using least squares estimation to estimate your β s, you can build upon that model in three primary ways, described in the upcoming sections:
 - Adding in modeling assumptions about the errors
 - Adding in more predictors
 - Transforming the predictors

Adding in modeling assumptions about the errors

- If you use your model to predict y for a given value of x , your prediction is deterministic and doesn't capture the variability in the observed data.
- From the Figure 3-5 that for a fixed value of $x=5$, there is variability among the time spent on the site.
- You want to capture this variability in your model, so you extend your model to:

$$y = \beta_0 + \beta_1 x + \epsilon$$

- Where the new term ϵ is referred to as noise.
- It's also called the **error term**— ϵ represents the actual error
- The difference between the observations and the true regression line, which you'll never know and can only estimate with your $\hat{\beta}$ s.

- One often makes the modeling assumption that the noise is normally distributed, which is denoted

$$\epsilon \sim N(0, \sigma^2)$$

- Sigma is the variance

With the preceding assumption on the distribution of noise, this model is saying that, for any given value of x , the conditional distribution of y given x is $p(y|x) \sim N(\beta_0 + \beta_1 x, \sigma^2)$.

So, for example, among the set of people who had five new friends this week, the amount of the time they spent on the website had a *normal distribution* with a mean of $\beta_0 + \beta_1 * 5$ and a variance of σ^2 , and you're going to estimate your parameters β_0, β_1, σ from the data.

How do you fit this model? How do you get the parameters β_0, β_1, σ from the data?

- So what can you do with your observed data to estimate the variance of the errors?
- Now that you have the estimated line, you can see how far away the observed data points are from the line itself, and you can treat these differences, also known as observed errors or residuals, as observations themselves, or estimates of the actual errors, the ϵ s.
- Why are we dividing by $n-2$? A natural question. Dividing by $n-2$, rather than just n , produces an unbiased estimator.

Define $e_i = y_i - \hat{y}_i = y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)$ for $i = 1, \dots, n$.

Then you estimate the variance (σ^2) of ϵ , as:

$$\frac{\sum_i e_i^2}{n-2}$$

- This is called the *mean squared error* and captures how much the predicted value varies from the observed.
- Mean squared error is a useful quantity for any prediction problem.
- It appears in the evaluation metrics in the following section.

Evaluation metrics

- You have a couple values in the output of the R function that help you get at the issue of how confident you can be in the estimates:
 - p-values and R-squared.
- Going back to our model in R, if we type in
`summary(model)`

which is the name we gave to this model, the output would be:

```
summary (model)
```

```
Call:
```

```
lm(formula = y ~ x)
```

```
Residuals:
```

Min	1Q	Median	3Q	Max
-121.17	-52.63	-9.72	41.54	356.27

```
Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-32.083	16.623	-1.93	0.0565 .
x	45.918	2.141	21.45	<2e-16 ***

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 77.47 on 98 degrees of freedom
```

```
Multiple R-squared: 0.8244,    Adjusted R-squared: 0.8226
```

```
F-statistic: 460 on 1 and 98 DF,  p-value: < 2.2e-16
```

- **R-squared**

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- This can be interpreted as the proportion of variance explained by our model.
- Note that mean squared error is in there getting divided by total error, which is the proportion of variance unexplained by our model and we calculate 1 minus that.

- **p-values**

- Looking at the output, the estimated β s are in the column marked Estimate.
- To see the p-values, look at $\Pr(>|t|)$.
- We can interpret the values in this column as follows:
 - We are making a null hypothesis that the β s are zero.
- For any given β ,
 - the p-value captures the probability of observing the data,
 - Obtaining the test-statistic that we obtained under the null hypothesis.
- This means that if we have a low p-value, it is highly unlikely to observe such a **test-statistic** under the null hypothesis, and the coefficient is highly likely to be nonzero and therefore significant.

- **Cross-validation**
- Another approach to evaluating the model is as follows.
- Divide our data up into a training set and a test set: 80% in the training and 20% in the test.
- Fit the model on the training set, then look at the ***mean squared error*** on the test set and compare it to that on the training set.
- Make this comparison across sample size as well. ***If the mean squared errors are approximately the same, then our model generalizes well and we're not in danger of over fitting.***
- See Figure 3-6 to see what this might look like. This approach is highly recommended.

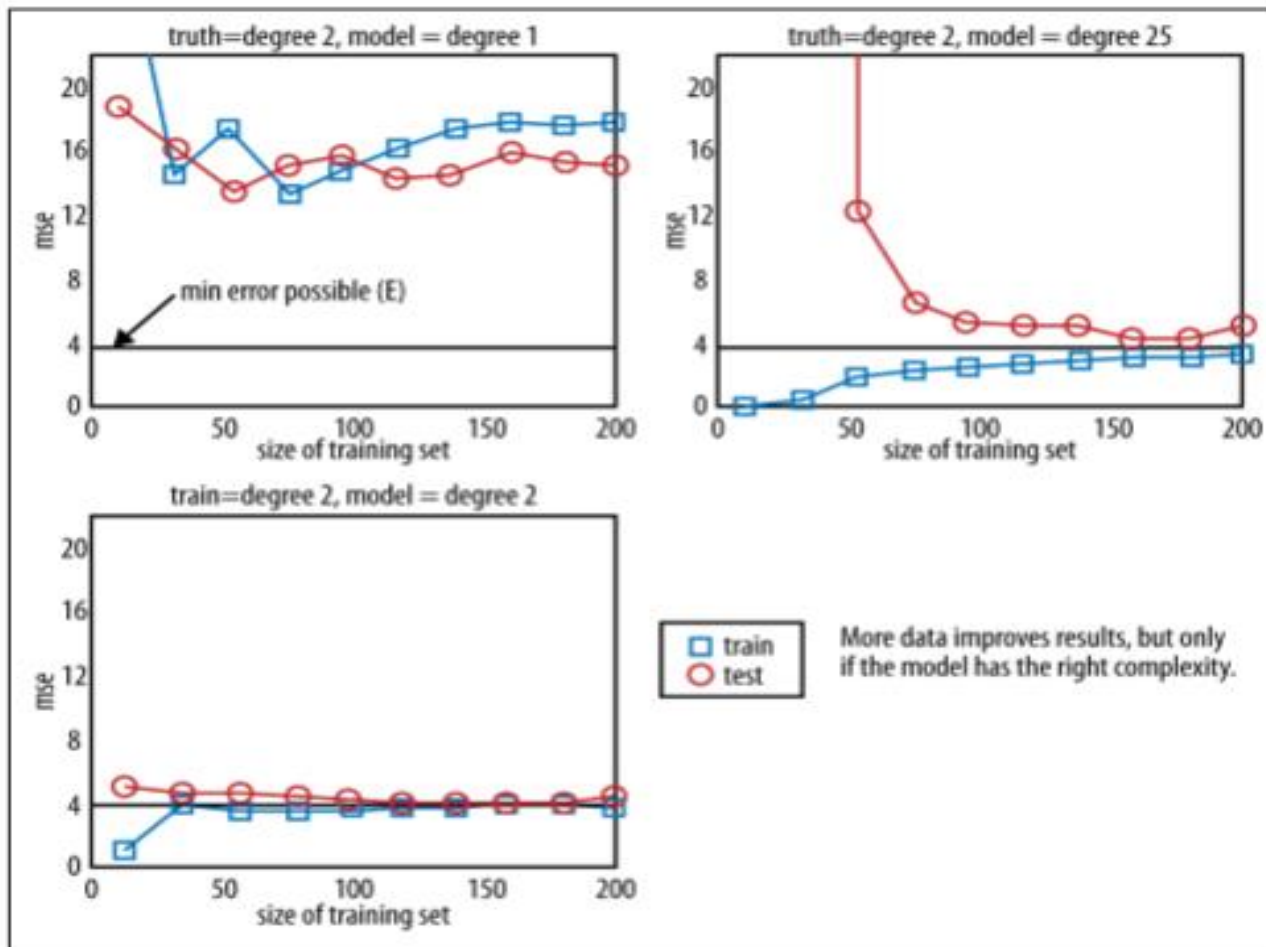


Figure 3-6. Comparing mean squared error in training and test set, taken from a slide of Professor Nando de Freitas; here, the ground truth is known because it came from a dataset with data simulated from a known distribution

Other models for error terms

- The mean squared error is an example of what is called a ***loss function***.
- This is the standard one to use in linear regression because it gives us a pretty nice measure of closeness of fit.
- It has the additional desirable property that by assuming that ε 's are normally distributed, we can rely on the ***maximum likelihood principle***. (Fitting a model)

- **Adding other predictors**
- What we just looked at was simple linear regression
 - —one outcome or dependent variable and one predictor.
- But we can extend this model by building in other predictors, which is called ***multiple linear regression***:
 $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \epsilon$.
- All the math that we did before holds because we had expressed it in matrix notation, so it was already generalized to give the appropriate estimators for the β .
- In the example we gave of predicting time spent on the website, the other predictors could be the user's age and gender, for example.

- The R code will just be:
- `model <- lm(y ~ x_1 + x_2 + x_3)` Or to add in interactions between variables:
- `model <- lm(y ~ x_1 + x_2+x_3+x2_*x_3)`

Transformations

- Going back to one x predicting one y , why did we assume a linear relationship? Instead, maybe, a better model would be a polynomial relationship like this:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$

- Wait, but isn't this linear regression? Last time we checked, **polynomials weren't linear**.
- To think of it as linear, **you transform or create new variables**—for example, $z = x^2$ — build a regression model based on z .

- If you look at the plot of
 - time spent *versus* number friends,
 - the shape looks a little bit curvy.
 - You could potentially explore this further by building up a model
 - and checking to see whether this yields an improvement.

```
# Simulating fake data
x_1 <- rnorm(1000,5,7) # from a normal distribution simulate
                        # 1000 values with a mean of 5 and
                        # standard deviation of 7
hist(x_1, col="grey") # plot p(x)
true_error <- rnorm(1000,0,2)
true_beta_0 <- 1.1
true_beta_1 <- -8.2

y <- true_beta_0 + true_beta_1*x_1 + true_error

hist(y) # plot p(y)
plot(x_1,y, pch=20,col="red") # plot p(x,y)
```

- Build a regression model and see that it recovers the true values of the β s.
- Simulate another fake variable x_2 that has a Gamma distribution with parameters you pick
- Now make the truth be that y is a linear combination of both x_1 and x_2 .
 - Fit a model that only depends on x_1 .
 - Fit a model that only depends on x_2 .
 - Fit a model that uses both.
- Vary the sample size and make a plot of **mean square error of the training set** and of the test set versus sample size.
- Create a new variable, z , that is equal to x_1^2 .
 - Include this as one of the predictors in your model.
 - See what happens when you fit a model that depends on x_1 only and then also on z .
- Vary the sample size and make a plot of mean square error of the training set and of the test set versus sample size.

- Play around more by
 - (a) changing parameter values (the true β s),
 - (b) changing the distribution of the true error, and
 - (c) including more predictors in the model with other kinds of probability distributions.
- `rnorm()` means randomly generate values from a normal distribution.
- `rbinom()` does the same for binomial. So look up these functions online and try to find more.
- Create scatterplots of all pairs of variables and histograms of single variables.

K-Nearest Neighbor

- *K-NN is an algorithm that can be used when you have a bunch of objects that have been classified or labeled in some way, and other similar objects that haven't gotten classified or labeled yet, and you want a way to automatically label them.*
- EX:- it could be patients who have been classified as “high cancer risk” or “low cancer risk.”
 - Can we use linear regression here ,the output is a **continuous variable**.
 - Here the output of your algorithm is going to be a **categorical label**, so linear regression wouldn't solve the problem as it's described.
 - not impossible to solve it with linear regression plus the concept of a “threshold.”

- Ex:- if you're trying to predict people's credit scores from their ages and incomes-
 - Then picked a threshold such as 700
 - A person whose age and income > 700 => credit as "high" OR toss bin labeled "high." Otherwise into the bin labeled "low."
 - With more thresholds, you could also have categories like "very low," "low," "medium," "high," and "very high."
- But not everything is on a continuous scale like a credit score.
- Ex:- what if your labels are "likely Democrat," "likely Republican," and "likely independent"?
 - What do you do now?
 - The intuition behind k-NN is to consider the most similar other items defined in terms of their attributes, look at their labels, and give the unassigned item the majority vote.

- To automate it, two decisions must be made:
 - how do you define similarity or closeness?
 - you can take the most similar items and call them neighbors.

Example with credit scores

- Considering age, income, and a credit category of high or low for a bunch of people and you want to use the **age and income** to predict the credit label of “high” or “low” for a new person.
- For example, here are the first few rows of a dataset, with income represented in thousands:

age	income	credit
69	3	low
66	57	low
49	79	low
49	17	low
58	26	high
44	71	high

You can plot people as points on the plane and label people with an empty circle if they have low credit ratings, as shown in **Figure 3-7**.

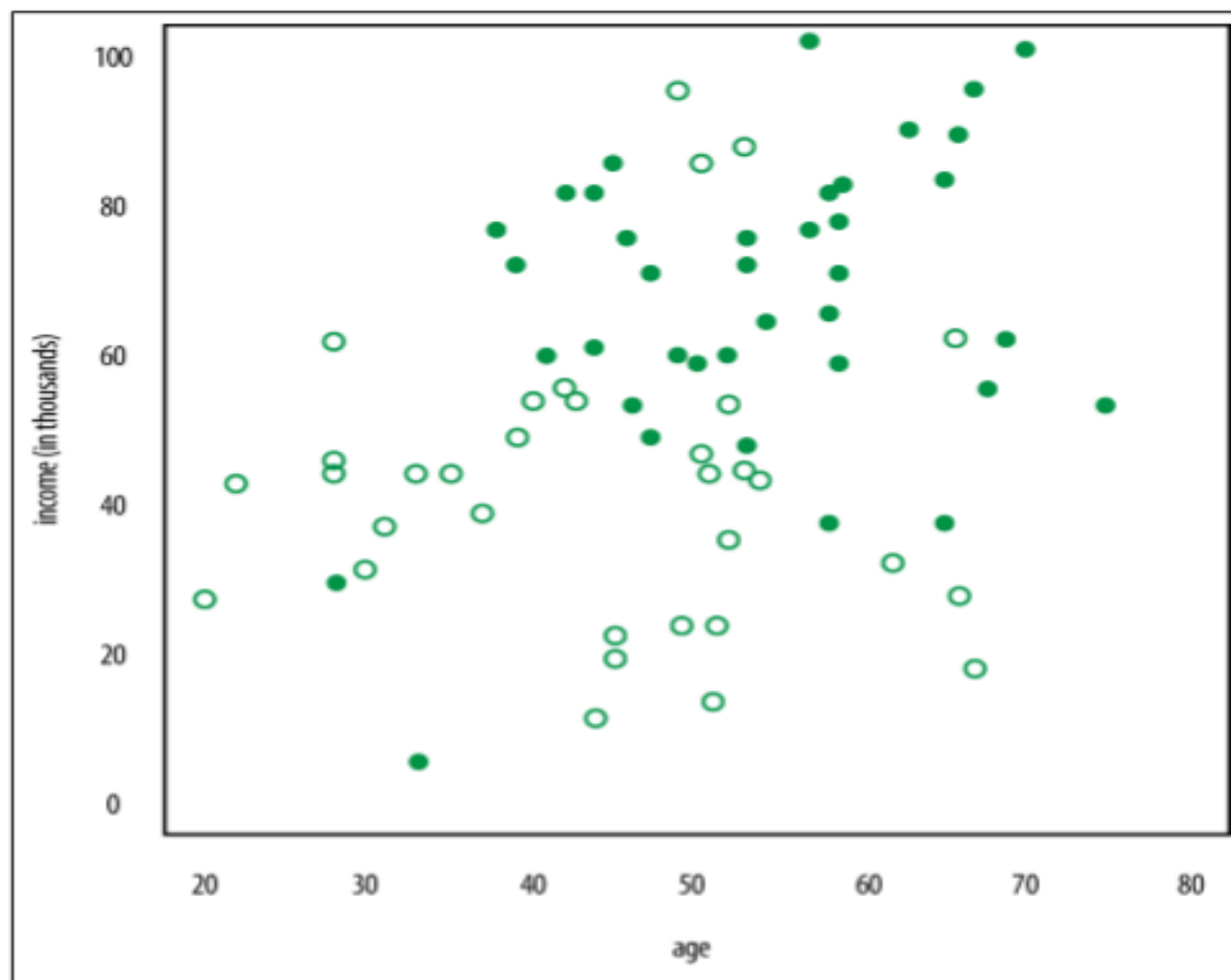


Figure 3-7. Credit rating as a function of age and income

What if a new guy comes in who is 57 years old and who makes \$37,000? What's his likely credit rating label? Look at **Figure 3-8**. Based on the other people near him, what credit score label do you think he should be given? Let's use k-NN to do it automatically.

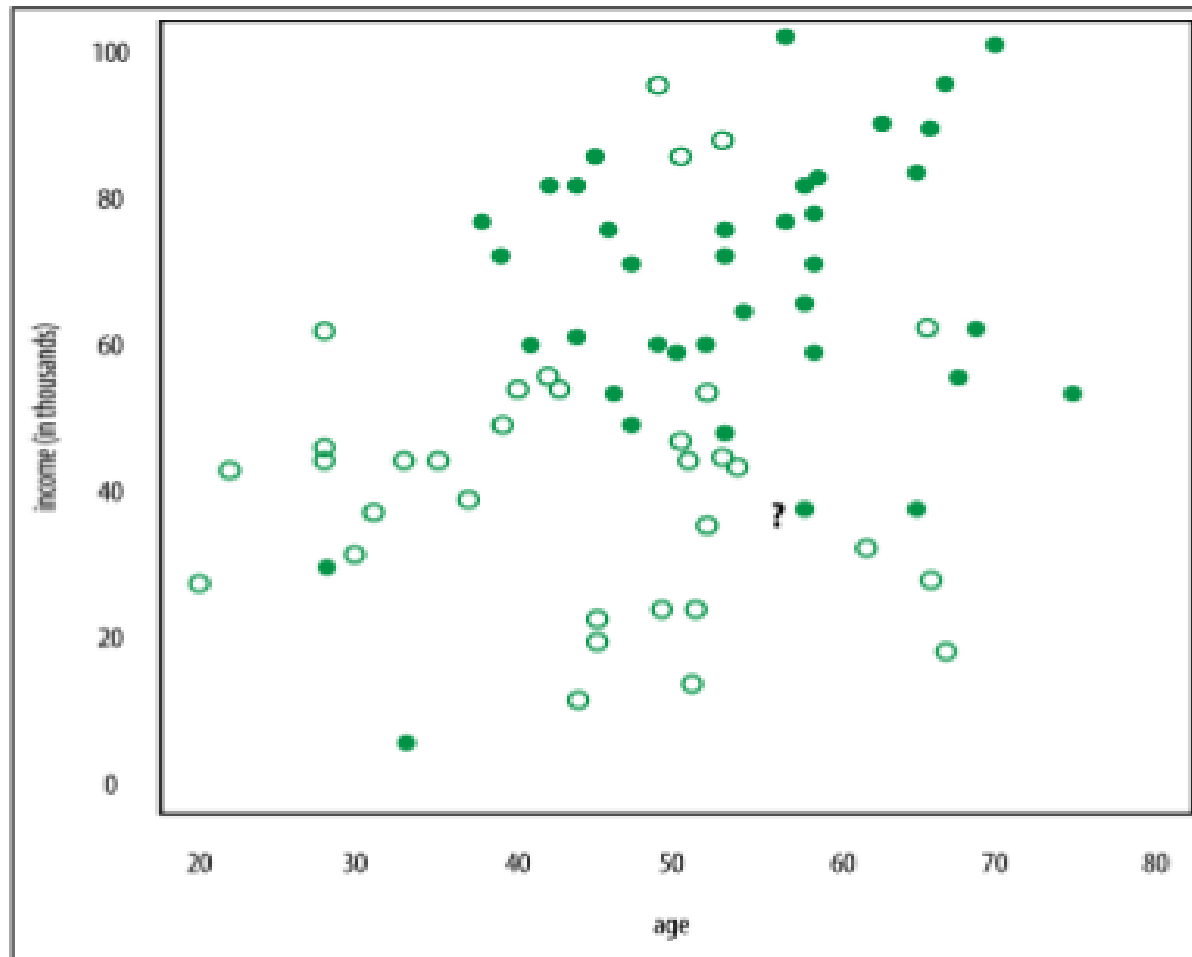


Figure 3-8. What about that guy?

- **Here's an overview of the process:**
- Decide on your similarity or distance metric.
- Split the original labeled dataset into training and test data.
- Pick an evaluation metric. (Misclassification rate is a good one. We'll explain this more in a bit.)
- Run k-NN a few times, changing k and checking the evaluation measure.
- Optimize k by picking the one with the best evaluation measure.
- Once you've chosen k, use the same training set and now create a new test set with the people's ages and incomes that you have no labels for, and want to predict. In this case, your new test set only has one lonely row, for the 57-year-old.

Similarity or distance metrics

- Definitions of “closeness” and similarity vary depending on the context:
 - closeness in social networks could be defined as the number of overlapping friends,
 - ex:- For the sake of our problem of what a neighbor is, we can use Euclidean distance on the plane if the variables are on the same scale.
- Euclidean distance is a good go-to distance metric for attributes that are real-valued and can be plotted on a plane or in multidimensional space.

Some others are:

- ***Cosine Similarity*** :-can be used b/w two real-valued vectors, x and y , and will yield a value between -1 (exact opposite) and 1 (exactly the same) with 0 in between meaning independent.

$$\cos(\vec{x}, \vec{y}) = \frac{\vec{x} \cdot \vec{y}}{\|\vec{x}\| \|\vec{y}\|}.$$

- ***Jaccard Distance or Similarity***:- gives the distance b/w a set of objects—

- Ex:-a list of Cathy's friends $A = \{\text{Kahn, Mark, Laura, ...}\}$

- a list of Rachel's friends $B = \{\text{Mladen, Kahn, Mark, ...}\}$

—and says how similar those two sets are

$$: J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

- ***Mahalanobis Distance*** :-can be used between two real-valued vectors and has the advantage over Euclidean distance that it takes into account correlation and is scale-invariant. where S is the covariance matrix.

$$d(\vec{x}, \vec{y}) = \sqrt{(\vec{x} - \vec{y})^T S^{-1} (\vec{x} - \vec{y})}$$

- ***Hamming Distance***:- Can be used to find the distance b/w two strings or pairs of words or DNA sequences of the same length.
 - The distance between olive and ocean is 4 because aside from the “o” the other 4 letters are different.
 - The distance between shoe and hose is 3 because aside from the “e” the other 3 letters are different.
 - You just go through each position and check whether the letters the same in that position, and if not, increment your count by 1.

- ***Manhattan*** :-This is also a distance between two real-valued k-dimensional vectors.
- The image to have in mind is that of a taxi having to travel the city streets of Manhattan.
- which is laid out in a grid-like fashion (you can't cut diagonally across buildings).
- The distance is therefore defined as
$$d(\vec{x}, \vec{y}) = \sum_i^k |x_i - y_i|,$$
- where i is the ith element of each of the vectors.

Putting it all together

- Now that you have a distance measure and an evaluation metric, you're ready to roll.
- For each person in your test set, you'll pretend you don't know his label.
- Look at the labels of his three nearest neighbors, say,
 - use the label of the majority vote to label him.
 - You'll label all the members of the test set and then use the misclassification rate to see how well you did.
 - All this is done automatically in R, with just this single line of R code:

```
knn (train, test, cl, k=3)
```

Choosing k

- How do you choose k?
- This is a parameter you have control over.
- You might need to understand your data pretty well to get a good guess.
- Then you can try a few different k's and see how your evaluation changes.
- So you'll run k-nn a few times, changing k, and checking the evaluation metric each time.

```

# we'll loop through and see what the misclassification rate
# is for different values of k
for (k in 1:20) {
  print(k)
  predicted.labels <- knn(train, test, cl, k)
  # We're using the R function knn()
  num.incorrect.labels <- sum(predicted.labels != true.labels)
  misclassification.rate <- num.incorrect.labels /
                           num.test.set.labels
  print(misclassification.rate)
}

```

Here's the output in the form (k, misclassification rate):

```

k  misclassification.rate
1, 0.28
2, 0.315
3, 0.26
4, 0.255
5, 0.23
6, 0.26
7, 0.25
8, 0.25
9, 0.235
10, 0.24

```

So let's go with $k=5$ because it has the lowest misclassification rate, and now you can apply it to your guy who is 57 with a \$37,000 salary. In the R console, it looks like:

```

> test <- c(57,37)
> knn(train,test,cl, k = 5)
[1] low

```

The output by majority vote is a low credit score when $k = 5$.

What are the modeling assumptions?

- The k-NN algorithm is an example of a nonparametric approach.
- You had no modeling assumptions(ex-probability distribution) about the underlying data-generating distributions, and you weren't attempting to estimate any parameters.
- But you still made some assumptions, which were:
 - Data is in some feature space where a **notion of “distance” makes sense.**
 - Training data has been labeled or classified into two or more classes.
 - You pick the ***number of neighbors*** to use, k.
 - You're assuming that the observed features and the labels are somehow associated.
 - They may not be, your evaluation metric **but will help you determine how good the algorithm is at labeling.**
 - You might want to add more features and check how that alters the evaluation metric.

But as always, you're in danger here of **over fitting**.

- Both linear regression and k-NN are examples of “**supervised learning**,” where you’ve observed both x and y , and you want to know the function that brings x to y .

k-means

- k-means is the first unsupervised learning technique, where the goal of the algorithm is **to determine the definition of the right answer by finding clusters of data** .
- Consider data at the user level, e.g., Google+ data, survey data, medical data, or SAT scores.
- Start by adding structure to your data. Assume each row of your dataset corresponds to a user as follows:

age gender income state household size

- Your goal is **to segment** the users. (stratify, group, or cluster the data.)

Why would you want to do this?

- Here are a few examples:
- You might want to give different users different experiences.
- Ex:-to offer toner to people who are known to **own printers**.
 - You might have a model that works better for specific groups.
 - Or you might have different models for different groups.
- ***Hierarchical modeling*** in statistics does something like this.

- Ex:-
 1. So for an attribute like age, you'd create bins: 20–24, 25–30, etc.
 2. Can be used for other attributes like income.
 3. States or cities are in some sense their own buckets.
 4. In that case, you could bucket the buckets and think of “East Coast” and “Midwest” or something like that.
 5. You may have 10 age buckets, 2 gender buckets, and so on, which would result in **$10 \times 2 \times 50 \times 10 \times 3 = 30,000$ possible bins**, which is big.
- Imagine this data existing in a **five-dimensional** space where each axis corresponds to one attribute.
- So there's a gender axis, an income axis, and so on.
- You can also label the various possible buckets along the corresponding axes, and if you did so, the resulting grid would consist of every possible bin
 - —a bin for each possible combination of attributes.

- Each user would then live in one of those 30,000 five-dimensional cells.
- But it's highly unlikely to build a different marketing campaign for each bin.
- So you'd have to bin the bins...
- Now you likely see the utility of having an algorithm to do this for you, especially if you could choose beforehand how many bins you want.
- **That's exactly what k-means is:**
 - a clustering algorithm where k is the number of bins.

2D version

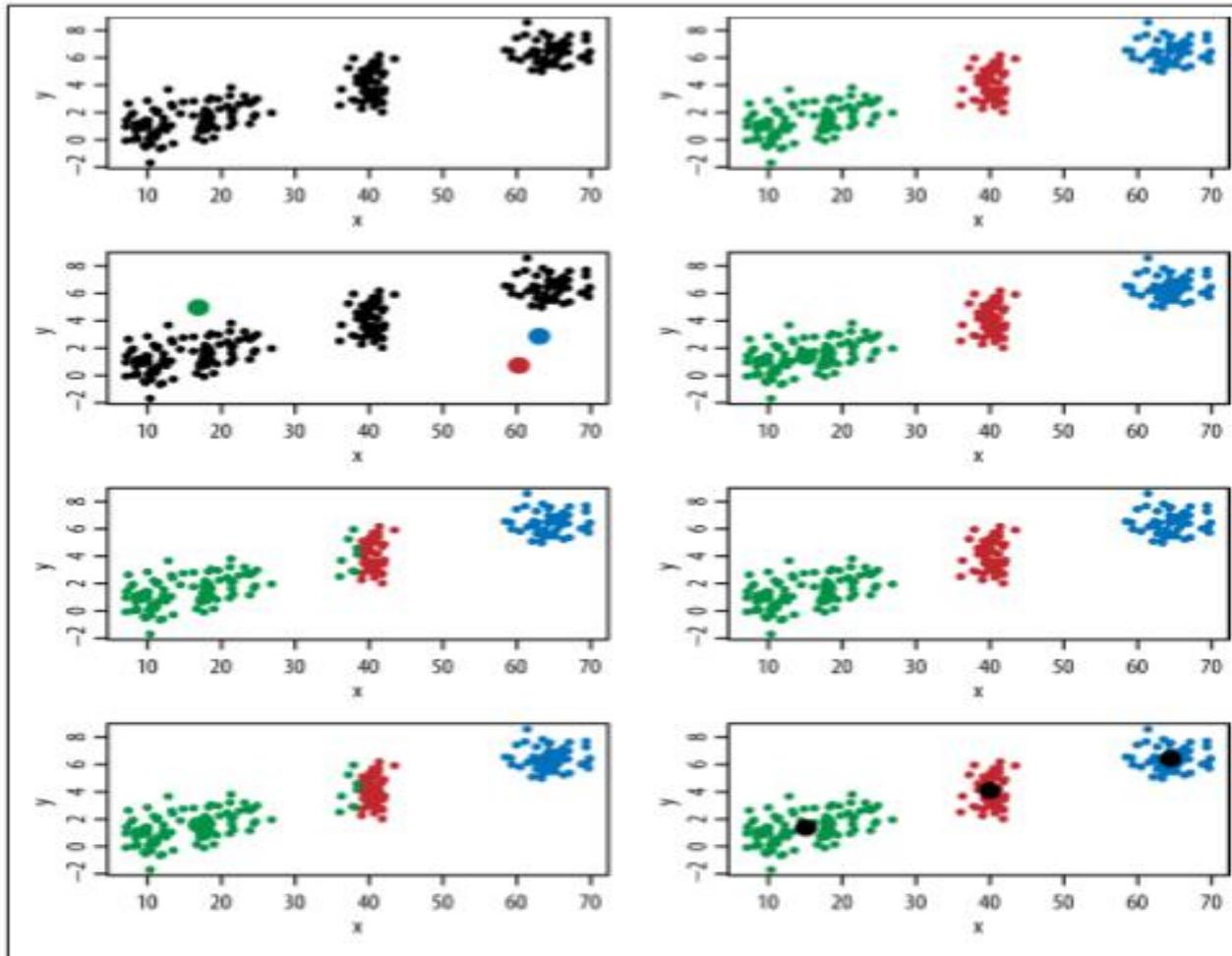


Figure 3-9. Clustering in two dimensions; look at the panels in the left column from top to bottom, and then the right column from top to bottom

- Consider a user ,how many adds you have shown to each user(the number of impressions) and how many times each has clicked on an ad.
- Top left the data naturally falls into cluster.
- Easy to see with eyes when 2D.
- When there are high dimensions and more data then algorithm are used in pattern finding process
- K mean looks for d dimensions,
D is the no of features for each data point.

- Here's how the algorithm illustrated in Figure 3-9 works:
- Initially, you randomly pick k centroids (or points that will be the center of your clusters) in d -space. Try to make them near the data but different from one another.
- Then assign each data point to the closest centroid.
- Move the centroids to the average location of the data points (which correspond to users in this example) assigned to it.
- Repeat the preceding two steps until the assignments don't change, or change very little.

- This is an example of unsupervised learning because the labels are not known and are instead discovered by the algorithm.
- k-means has some known issues:
 - Choosing k is more an art than a science, although there are bounds: $1 \leq k \leq n$, where n is number of data points.
 - There are convergence issues—the solution can fail to exist, if the algorithm falls into a loop, for example, and keeps going back and forth between two possible solutions, or in other words, there isn't a single unique solution.
 - Interpretability can be a problem—sometimes the answer isn't at all useful. Indeed that's often the biggest problem.

- In spite of these issues, it's pretty fast (compared to other clustering algorithms), and there are broad applications in marketing, computer vision (partitioning an image), or as a starting point for other models.
- In practice, this is just one line of code in R:

```
kmeans(x, centers, iter.max = 10, nstart = 1, algorithm =  
c("Hartigan-Wong", "Lloyd", "Forgy", "MacQueen"))
```

Your dataset needs to be a matrix, *x*, each column of which is one of your features.

- You specify *k* by selecting centers. It defaults to a certain number of iterations, which is an argument you can change.
- You can also select the specific algorithm it uses to discover the clusters.

THANKYOU

DSBDA Unit 1 Part 1

Course Outcomes

- On Successful completion of this course, student will be able to:
- Understands various phases of the data analytics life cycle.
- Apply statistical methods to data for inferences.
- Analyze data using Classification, Graphical and computational methods.
- Understands Big Data technologies and NOSQL.
- Analyze various types of data using Data Analytics Techniques.

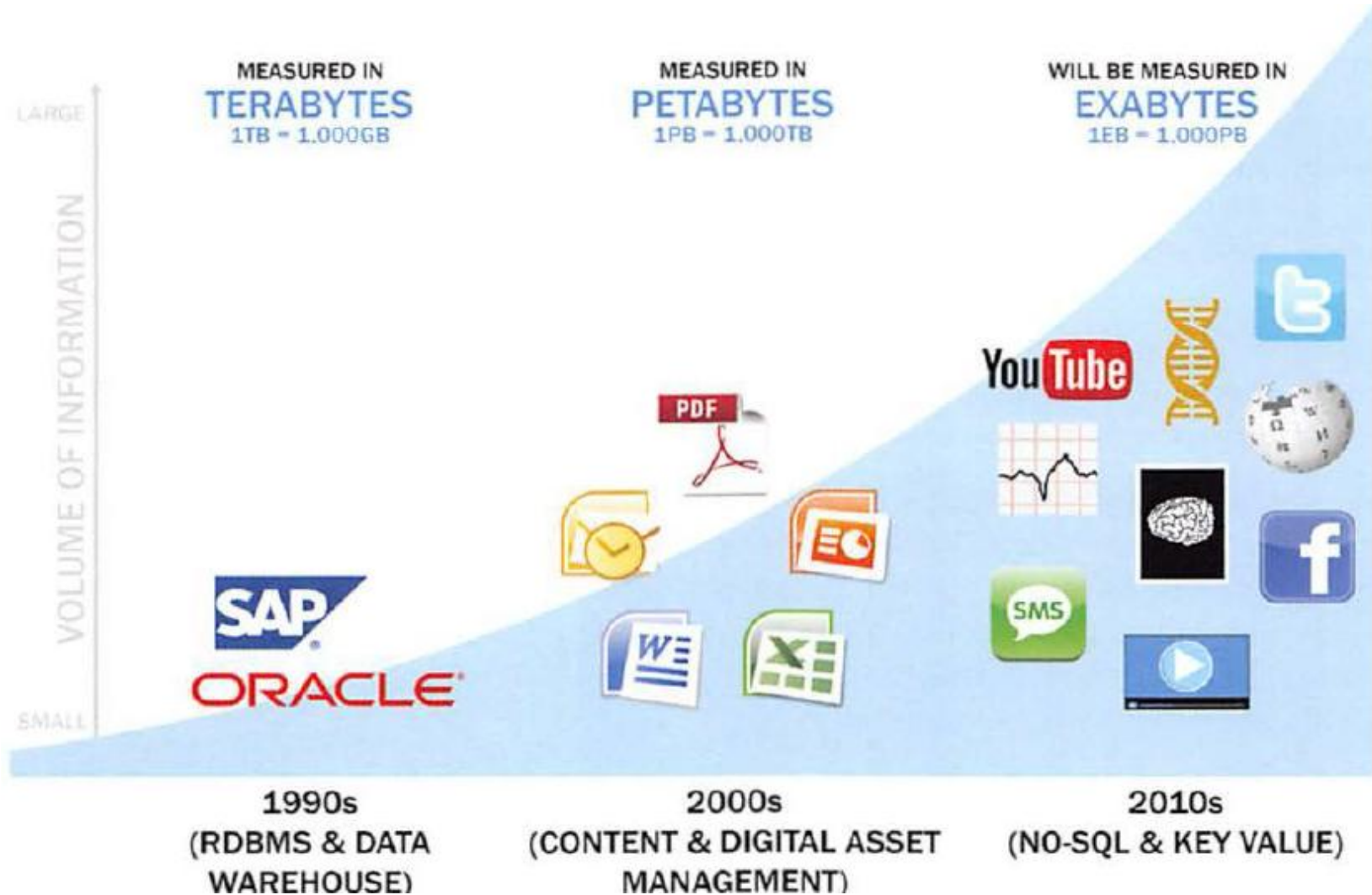
UNIT-I

- **Data Analytics Life cycle:** Data Analytics Life cycle Overview, Discovery, Data Preparation, Model Planning, Model Building, Communicate Results, Operationalize. Exploratory Data Analysis, Statistical Methods for Evaluation, ANOVA.

Overview of Bigdata

- Data is created constantly via Mobile phones, social media etc.
- Devices and sensors automatically generate diagnostic information that needs to be stored and processed in real time.
- Challenging is analyzing vast amounts of it, especially when it does not conform to traditional notions of data structure, to identify meaningful patterns and extract useful information.

Data evolution and the rise of Big Data sources



Big Data characteristics

- Huge volume of data: Rather than thousands or millions of rows, Big Data can be billions of rows and millions of columns.
- Complexity of data types and structures: Big Data reflects the variety of new data sources, formats, and structures.
- Speed of new data creation and growth: Big Data can describe high velocity data, with rapid data.

Note : Social media and genetic sequencing are among the fastest-growing sources of Big Data and examples of untraditional sources of data being used for analysis.

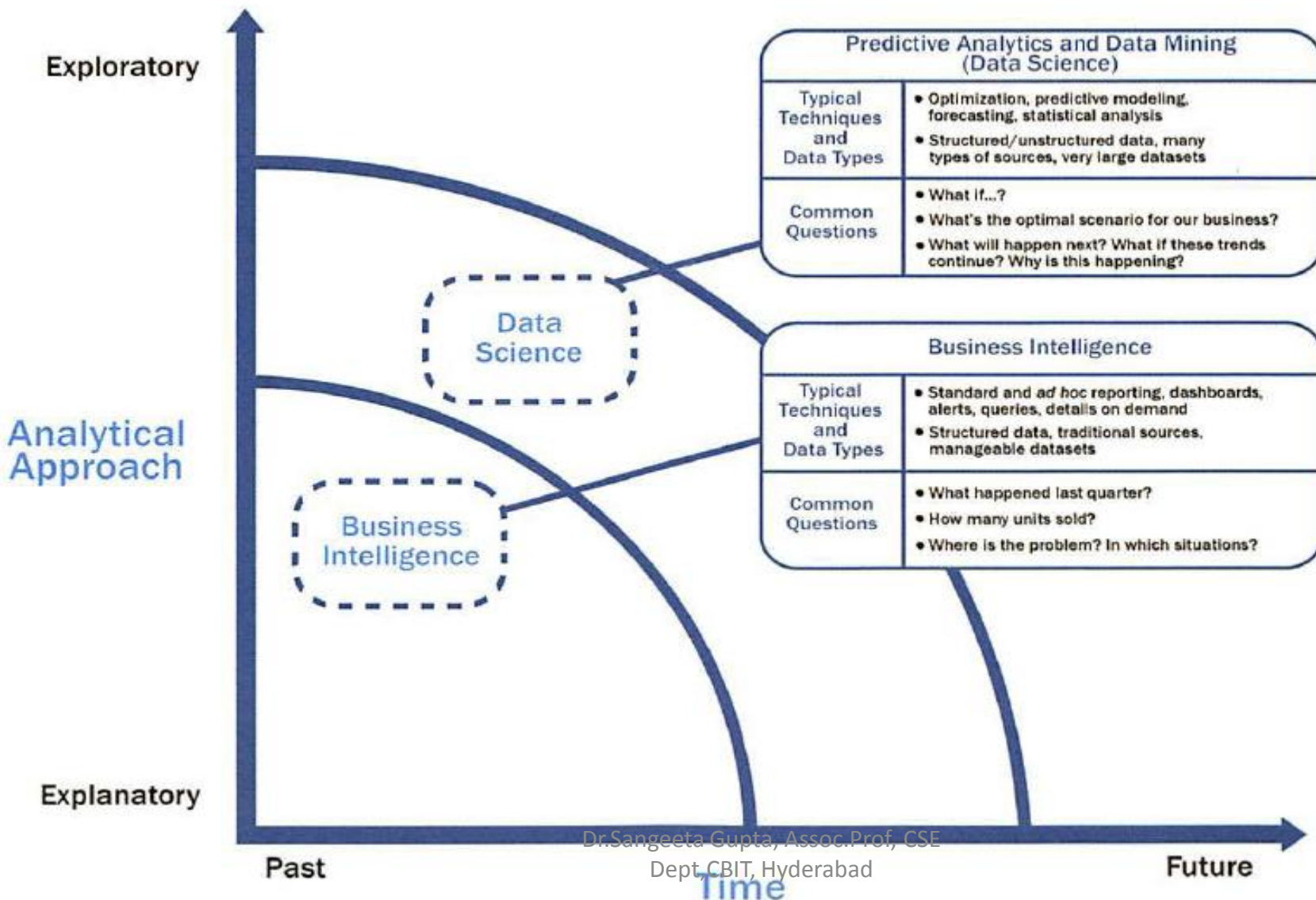
Data Structures

- Big data can come in multiple forms, including structured and non-structured data such as financial data, text files, multimedia files.
- requires different techniques and tools to process and analyze.
- Distributed computing environments and massively parallel processing (MPP) architectures that enable parallelized data ingest and analysis are the preferred approach to process such complex data.

4 Types of Data Structures

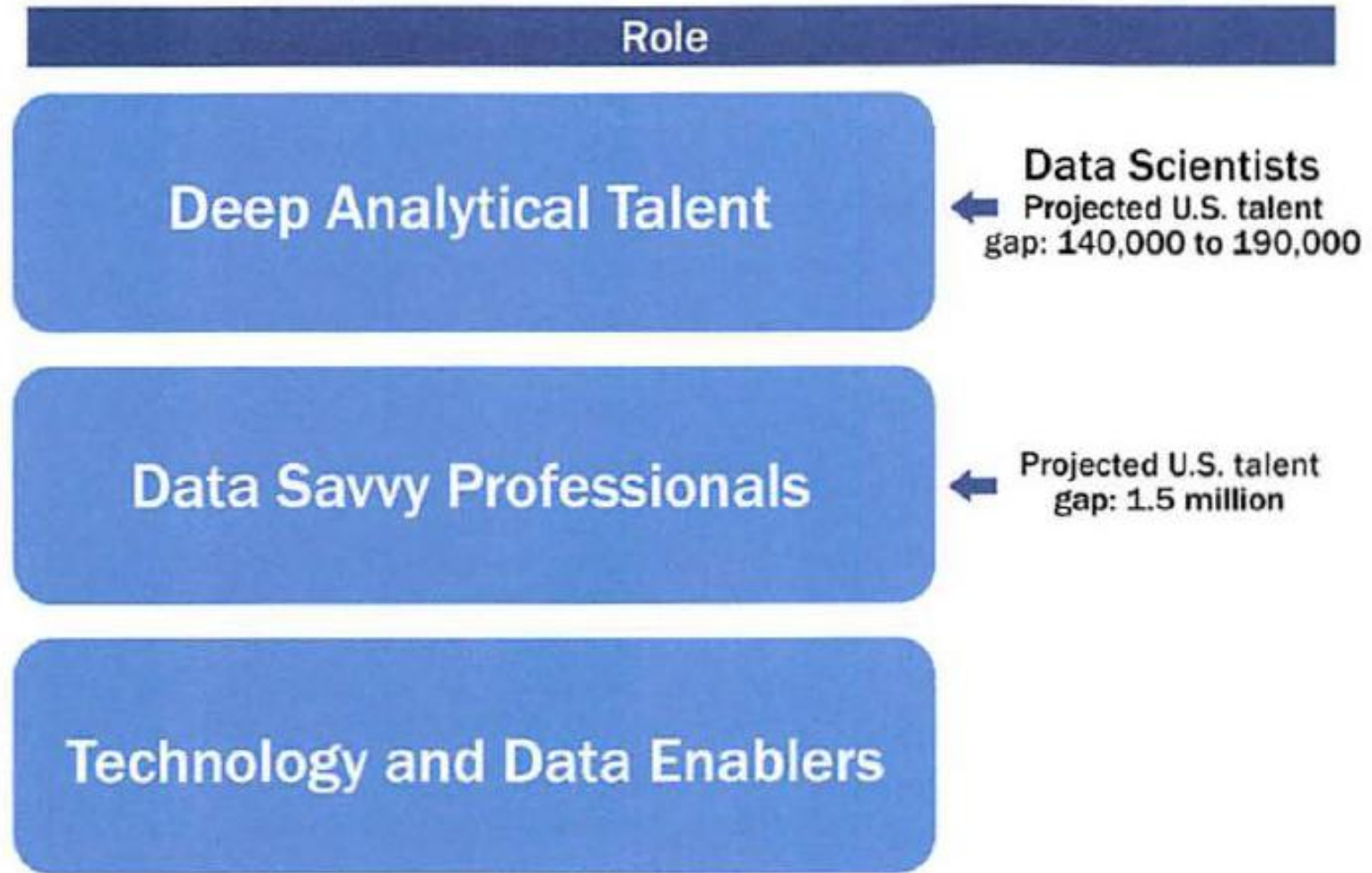
- **Structured data:** Data containing a defined data type, format, and structure that is, transaction data, online analytical processing (OLAP) data cubes, traditional RDBMS, CSV file, simple spreadsheets.
- **Semi-structured data:** Textual data files with a discernible pattern that enables parsing such as Extensible Markup Language [XML] data files that are self-describing and defined by an XML schema.
- **Quasi-structured data:** Textual data with erratic data (unpredictable) formats that can be formatted with effort, tools, and time (for instance, web clickstream data that may contain inconsistencies in data values and formats).
- **Unstructured data:** Data that has no inherent structure, which may include text documents, PDFs, images, and video.

BI Versus Data Science



Key Roles of Bigdata Ecosystem

Three Key Roles of The New Data Ecosystem



Note: Figures above reflect a projected talent gap in US in 2018, as shown in McKinsey May 2011 article "Big Data: The Next Frontier for Innovation, Competition, and Productivity"

- **Deep Analytical Talent-** is technically savvy, with strong analytical skills. Ex: Data Scientists.
- **Data Savvy Professionals-**has less technical depth but has a basic knowledge of statistics or machine learning and can define key questions that can be answered using advanced analytics. Ex : Financial Analyst.
- **Technology and Data Enablers.** This group represents people providing technical expertise to support analytical projects, such as provisioning and administering analytical sandboxes, and managing large-scale data architectures. Ex: database administrator.
- These three groups must work together closely to solve complex Big Data challenges

Thankyou

Big Data Analytics

Introduction to Hadoop



Dr. Satishkumar L. Varma

Professor, Department of Computer Engineering

PCE, New Panvel

www.sites.google.com/view/vsat2k

www.vsat2k.wordpress.com

www.vsat2k.moodlecloud.com

Course: Big Data Analytics

Module 1: Introduction to Big data

Module 2 : Introduction to Hadoop

Module 3 : NoSQL

Module 4: MapReduce and New S/w Stack

Module 5 : Finding similar Items

Module 6 : Mining Data Stream

Module 7 : Link Analysis

Module 8: Frequent item set

Module 9 : Clustering

Module 10 : Recommendation system

Module 11: Mining Social-Networks Graphs

2. Introduction to Hadoop

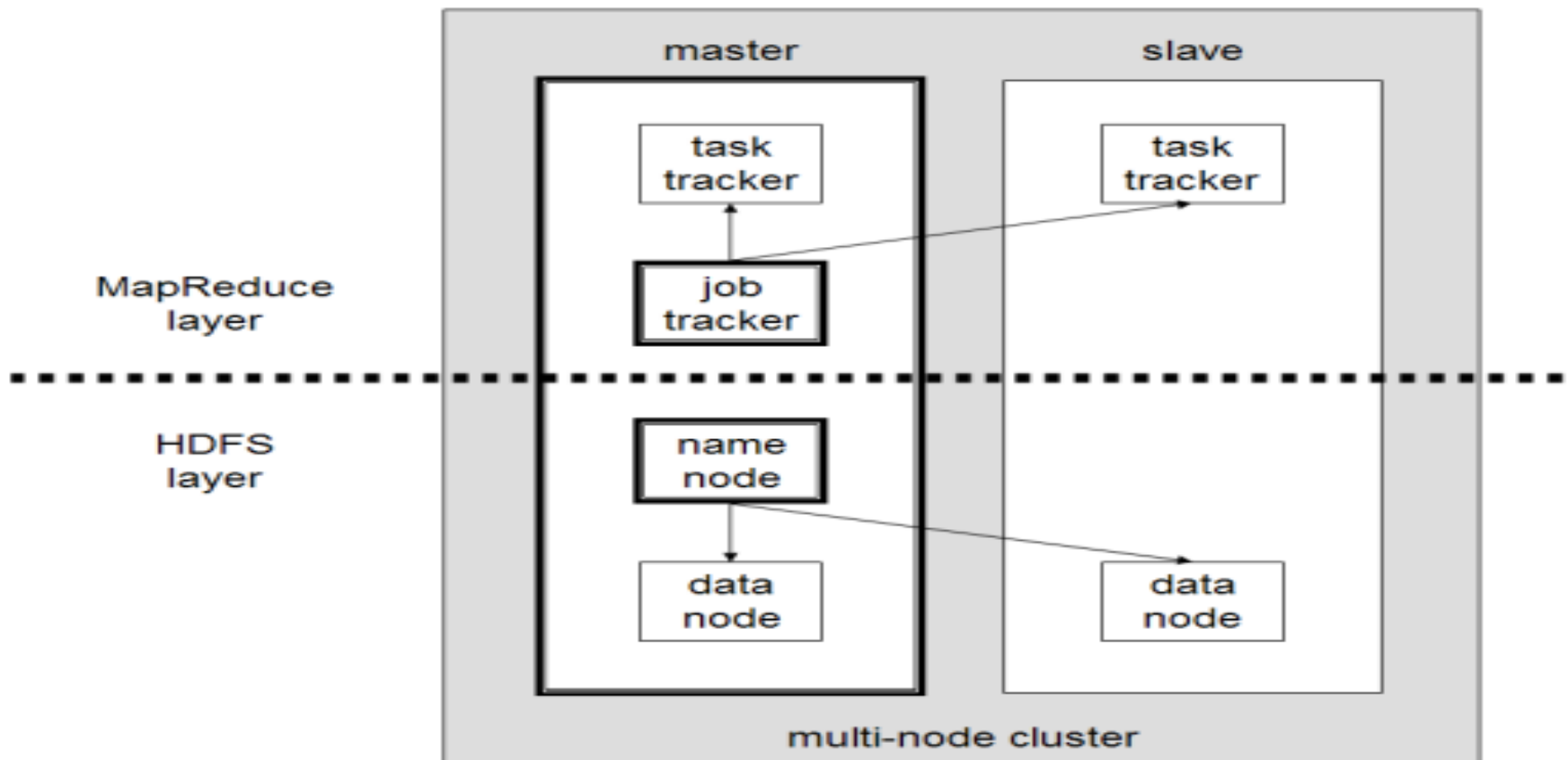
-  What is Hadoop? Definition
-  Goals / Requirements
-  Architecture Overview
-  Core Hadoop Components
-  Hadoop Ecosystem
-  Hadoop Limitations
-  Hadoop fs Shell Commands Examples

What is Hadoop?

- ❖ Hadoop is an apache open source software framework
- ❖ It is a java framework
- ❖ 4 distributed processing of **large datasets** across **large clusters** of nodes
 - ❖ **Large datasets:** Terabytes or petabytes of data
 - ❖ **Large clusters:** hundreds of thousands of computers (nodes)
- ❖ Hadoop is open-source implementation for Google **MapReduce**
- ❖ Hadoop is based on a simple programming model called **MapReduce**
- ❖ Runs on large cluster of commodity machines
- ❖ Hadoop provides both distributed storage and processing of large datasets
- ❖ Support processing on streaming data

What is Hadoop?

- ❖ **Hadoop framework consists on two main layers**
 - ❖ Distributed file system (HDFS)
 - ❖ Execution engine (MapReduce)



Why: Goals / Requirements?

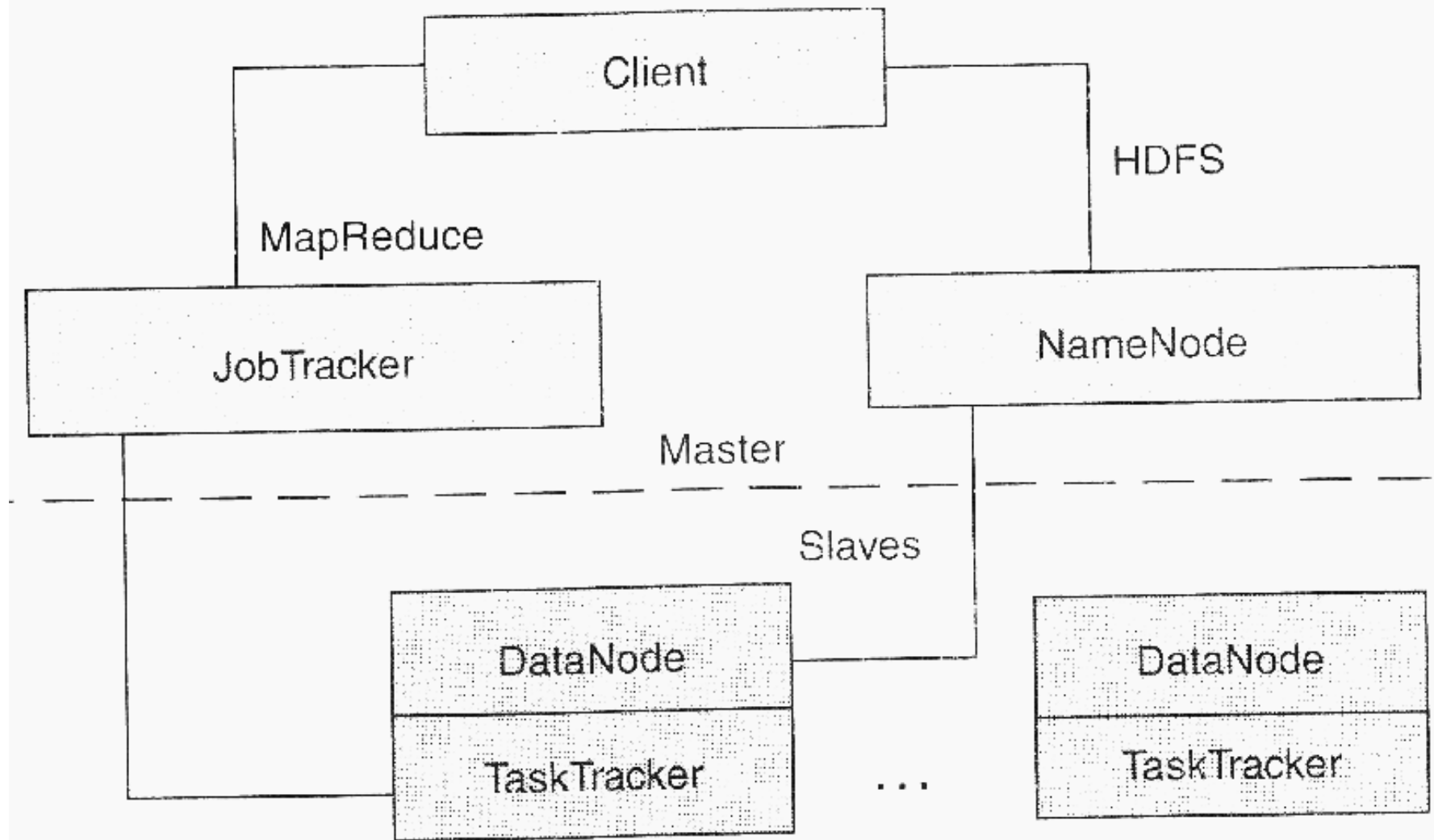
- ❄ Facilitate storage & processing of large and rapidly growing dataset
 - ❄ Structured and non-structured data
 - ❄ Simple programming models
- ❄ High scalability and availability
- ❄ Use commodity (cheap!) hardware with little redundancy
- ❄ Fault-tolerance
- ❄ Move computation rather than data

Hadoop Design Principles

- ☠ Need to process big data
- ☠ High Need to parallelize computation across thousands of nodes
- ☠ Use commodity (cheap!) hardware with little redundancy
- ☠ Contrast to **Parallel DBs**: Small number of high-end expensive machines
- ☠ **Commodity hardware**: Large number of low-end cheap machines working in parallel to solve a computing problem
- ☠ **Automatic parallelization & distribution**: Hidden from the end-user
- ☠ **Fault tolerance and automatic recovery**: Nodes/tasks will fail and will recover automatically
- ☠ **Clean and simple programming abstraction**: Users only provide two functions “map” and “reduce”

Hadoop Architecture?

❖ Hadoop Architecture



Hadoop Architecture Overview?

- 🔴 Hadoop architecture is a master/slave architecture
- 🌸 The master being the namenode and slaves are datanodes
- 🌸 Namenode controls the access to the data by clients
- 🌿 Datanodes manage the storage of data on the nodes that are running on.
- 🌿 Hadoop splits the file into one or more blocks and these blocks are stored in the datanodes
- 🌿 Each data block is replicated to 3 different datanodes to provide high availability of the hadoop system
- 🌿 The block replication factor is configurable

Who Uses MapReduce/Hadoop?

- ❁ Google: Inventors of MapReduce computing paradigm
- ❁ Yahoo: Developing Hadoop open-source of MapReduce
- ❁ IBM, Microsoft, Oracle
- ❁ Facebook, Amazon, AOL, NetFlex
- ❁ Many others + universities and research labs

Hadoop Components?

 Hadoop Distributed File System

 MapReduce Engine

 Types of Nodes

 Namenode

 Secondary Namenode

 Datanode

 JobTracker

 TaskTracker

 YARN – (Yet Another Resource Negotiator)

 Provides resource management for the processes running on Hadoop

Hadoop Components?

❖ Hadoop Distributed File System:

- ❖ HDFS is designed to run on commodity machines which are of low cost hardware
- ❖ Distributed data is stored in the HDFS file system
- ❖ HDFS is highly fault tolerant
- ❖ HDFS provides high throughput access to the applications that require big data
- ❖ Java-based scalable system that stores data across multiple machines without prior organization

Hadoop Components?

Namenode:

-  Namenode is the heart of the hadoop system
-  Namenode manages the file system namespace
-  It stores the metadata information of the data blocks
-  This metadata is stored permanently on to local disk in the form of namespace image and edit log file
-  Namenode also knows the location of the data blocks on data node
-  However the namenode does not store this information persistently
-  Namenode creates block to datanode mapping when it is restarted
-  If the namenode crashes, then the entire hadoop system goes down

Hadoop Components?

Secondary Namenode:

-  Responsibility is to periodically copy and merge the namespace image and edit log
-  In case if the name node crashes,
 -  Then the namespace image stored in secondary namenode can be used to restart the namenode.

Hadoop Components?

DataNode:

-  Stores the blocks of data and retrieves them
-  Datanodes also reports the blocks information to the namenode periodically
-  Stores the actual data in HDFS
-  Notifies NameNode of what blocks it has
-  Can run on any underlying filesystem (ext3/4, NTFS, etc)
-  NameNode replicates blocks 2x in local rack, 1x elsewhere

Hadoop Components?

JobTracker

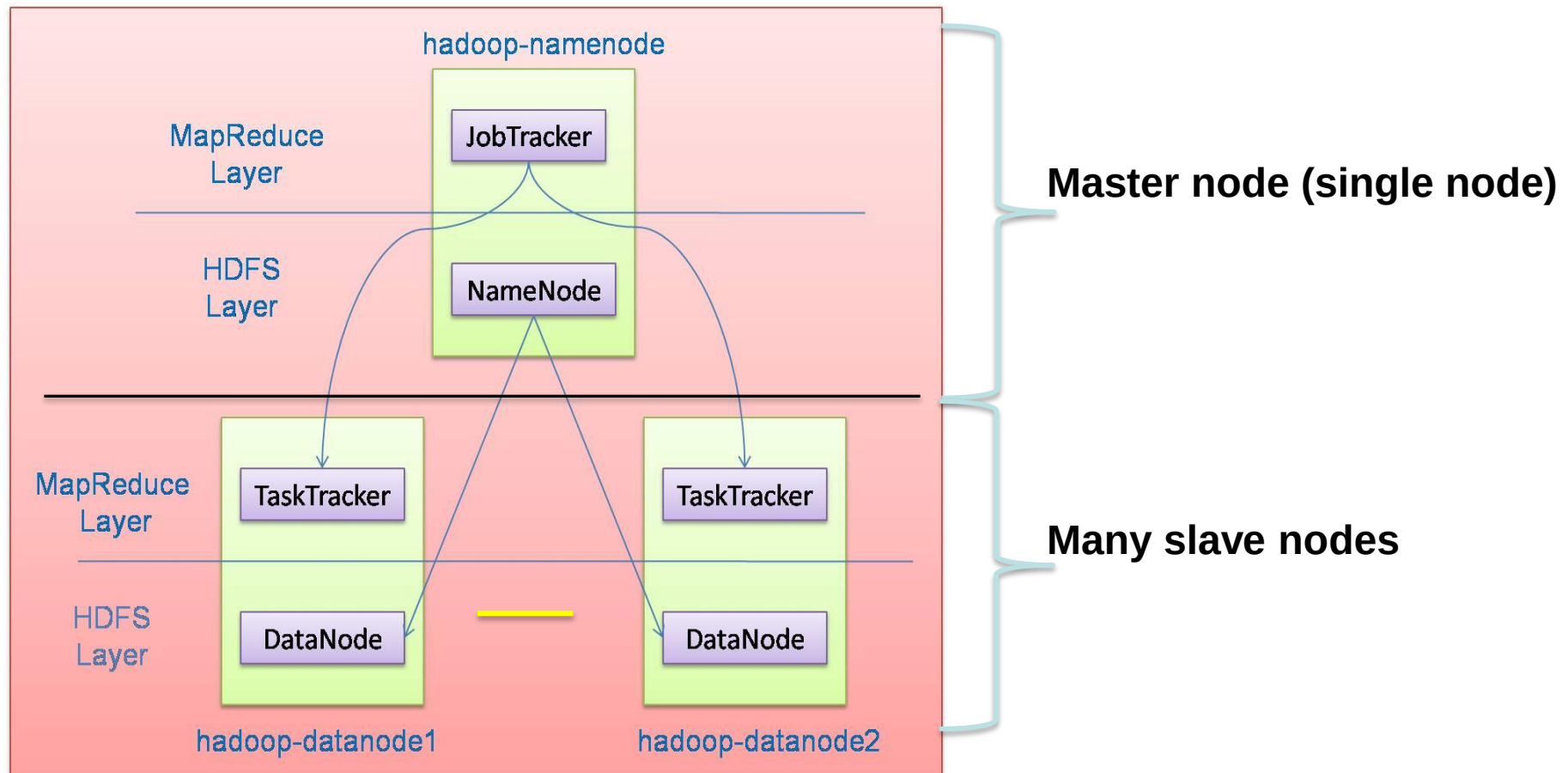
-  JobTracker responsibility is to schedule the clients jobs
-  Job tracker creates map and reduce tasks and schedules them to run on the datanodes (tasktrackers)
-  Job Tracker also checks for any failed tasks and reschedules the failed tasks on another datanode
-  Jobtracker can be run on the namenode or a separate node

TaskTracker

-  Tasktracker runs on the datanodes
-  Task trackers responsibility is to
 -  run the map or reduce tasks assigned by the namenode and
 -  report the status of the tasks to the namenode

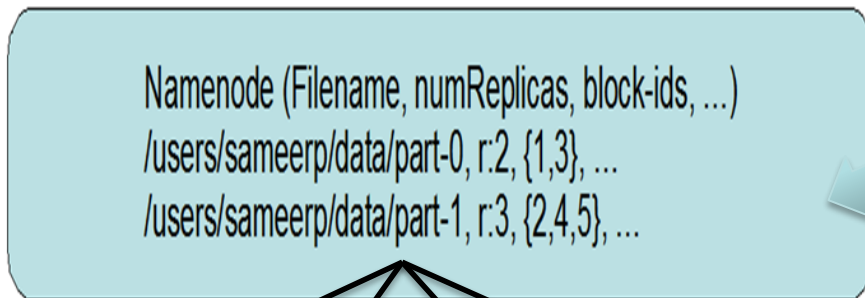
How does it work: Hadoop Architecture?

- 🌸 Hadoop distributed file system (HDFS)
- 🌸 MapReduce execution engine



Hadoop distributed file system (**HDFS**)

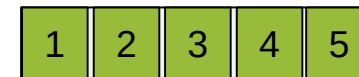
Block Replication



Centralized namenode

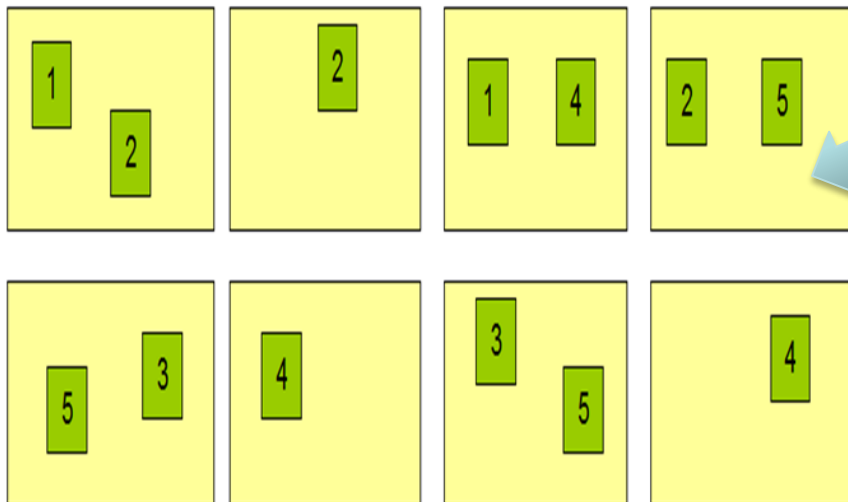
- Maintains metadata info about files

File *F*



Blocks (64/128 MB)

Datanodes



Many datanode (1000s)

- Store the actual data
- Files are divided into blocks
- Each block is replicated *N* times (Default = 3)

HDFS Properties

Large

-  A HDFS instance may consist of thousands of server machines, each storing part of the file system's data

Replication

-  Each data block is replicated many times (default is 3)

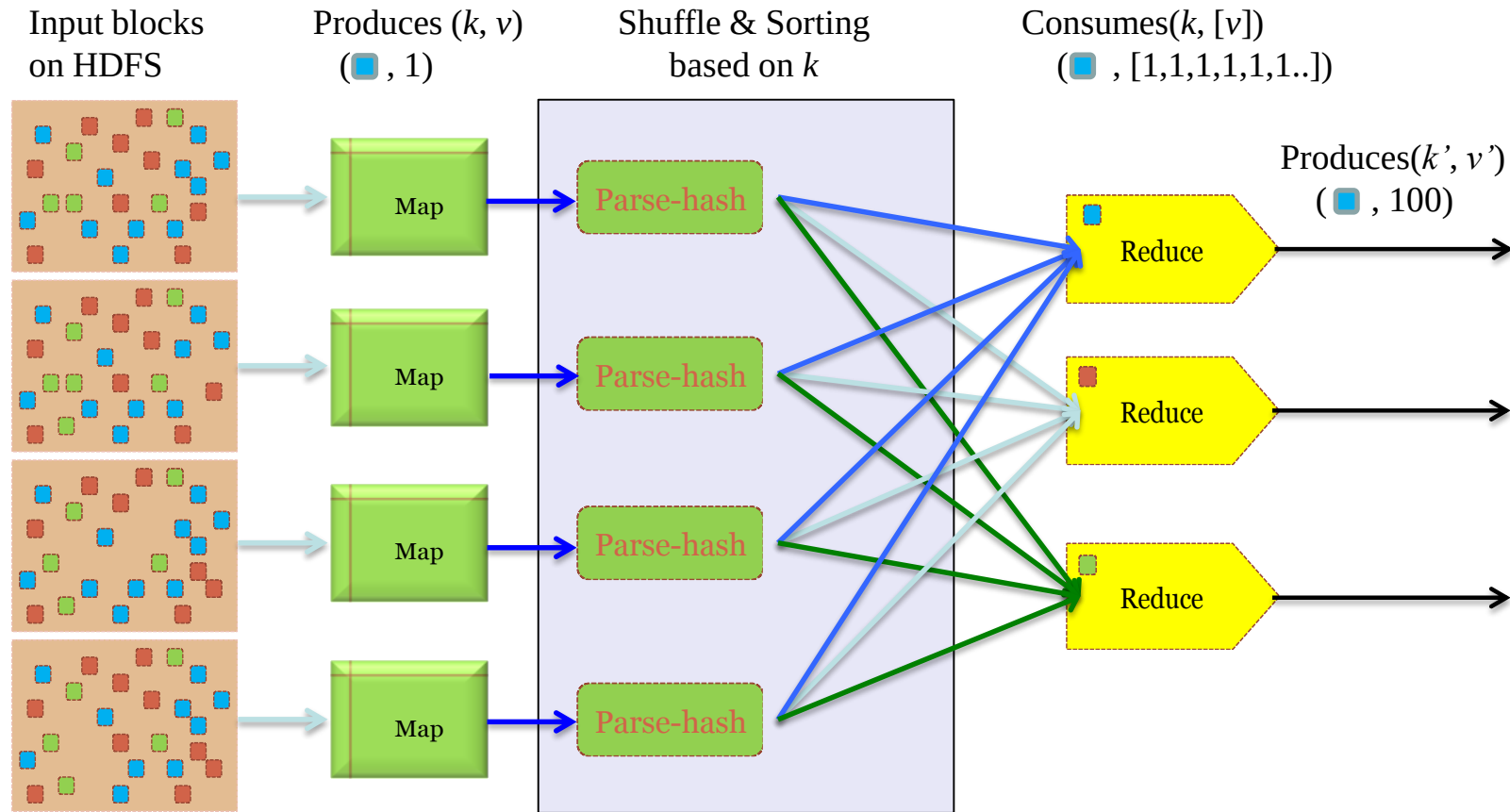
Failure

-  Failure is the norm rather than exception

Fault Tolerance

-  Detection of faults and quick, automatic recovery from them is a core architectural goal of HDFS

Map-Reduce Execution Engine: Color Count Example



Users only provide the “Map” and “Reduce” functions

Map-Reduce Engine

MapReduce Engine

 JobTracker

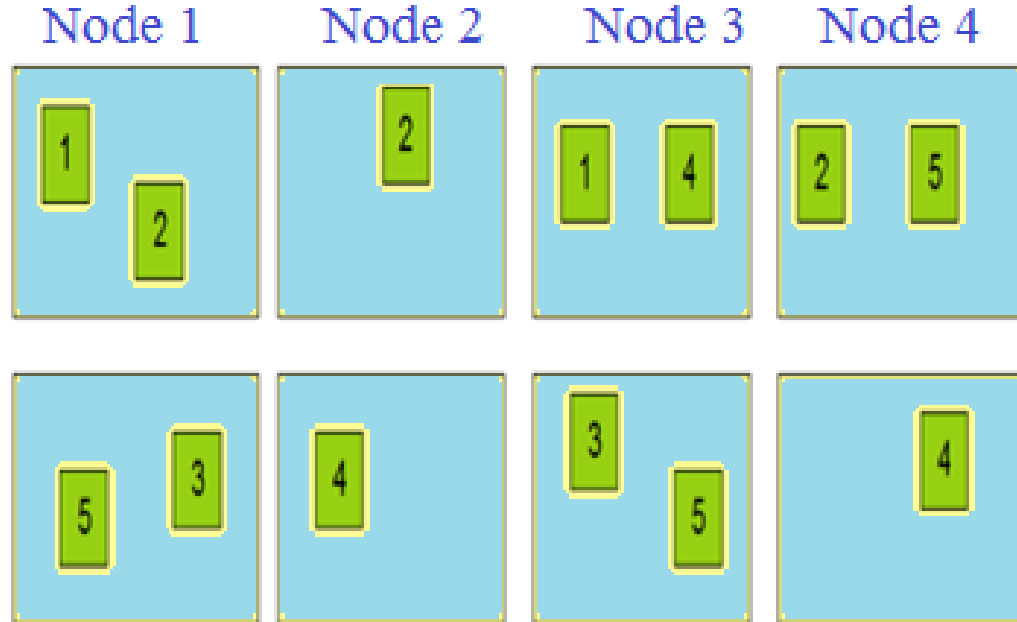
 TaskTracker

 JobTracker splits up data into smaller tasks(“Map”) and sends it to the TaskTracker process in each node

 TaskTracker reports back to the JobTracker node and reports on job progress, sends data (‘‘Reduce’’) or requests new jobs

MapReduce Engine Properties

- ❖ **Job Tracker is the master node (runs with the namenode)**
 - Receives the user's job
 - Decides on how many tasks will run (number of mappers)
 - Decides on where to run each mapper (concept of locality)
- ❖ This file has 5 Blocks → run 5 map tasks
- ❖ Where to run the task reading block “1”
 - ❖ *Try to run it on Node 1 or Node 3*

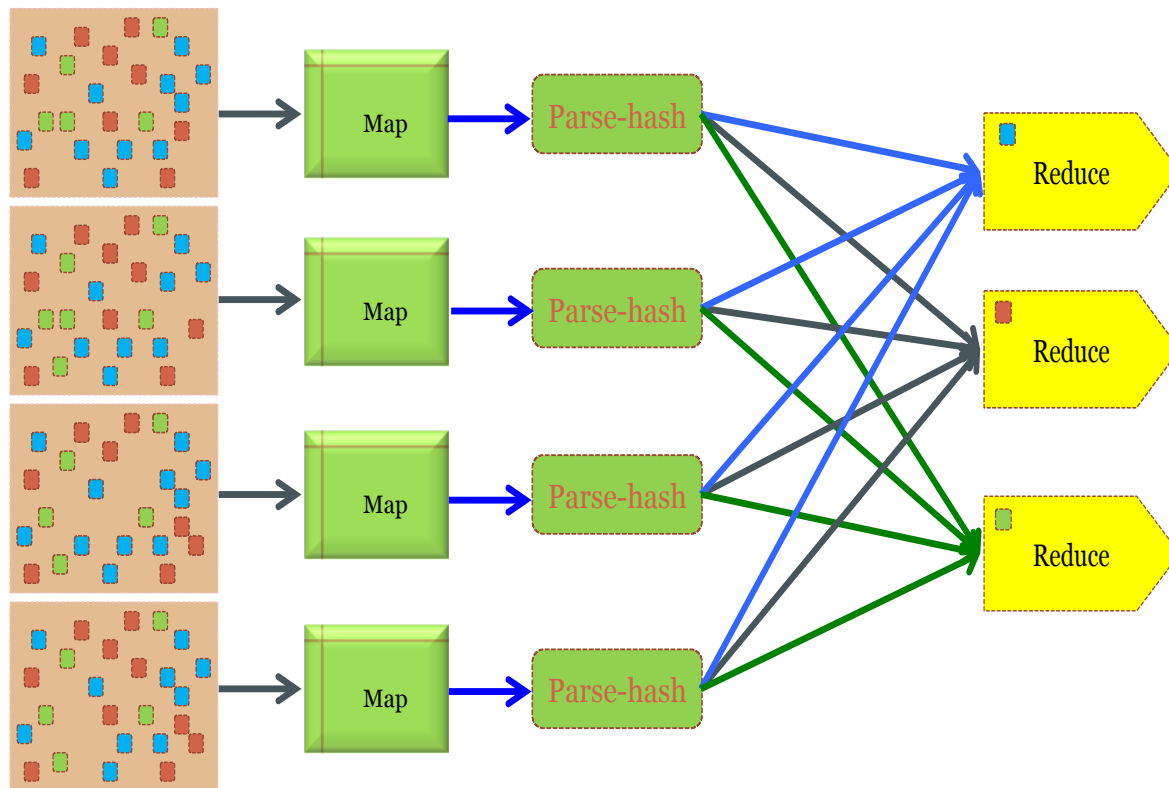


MapReduce Engine Properties

❖ Task Tracker is the slave node (runs on each datanode)

- Receives the task from Job Tracker
- Runs the task until completion (either map or reduce task)
- Always in communication with the Job Tracker reporting progress

❖ In this example, 1 map-reduce job consists of 4 map tasks and 3 reduce tasks



Key – Value Pair

- ❖ Mappers and Reducers are users' code (provided functions)
- ❖ Just need to obey the Key-Value pairs interface

❖ **Mappers:**

- Consume <key, value> pairs
- Produce <key, value> pairs

❖ **Reducers:**

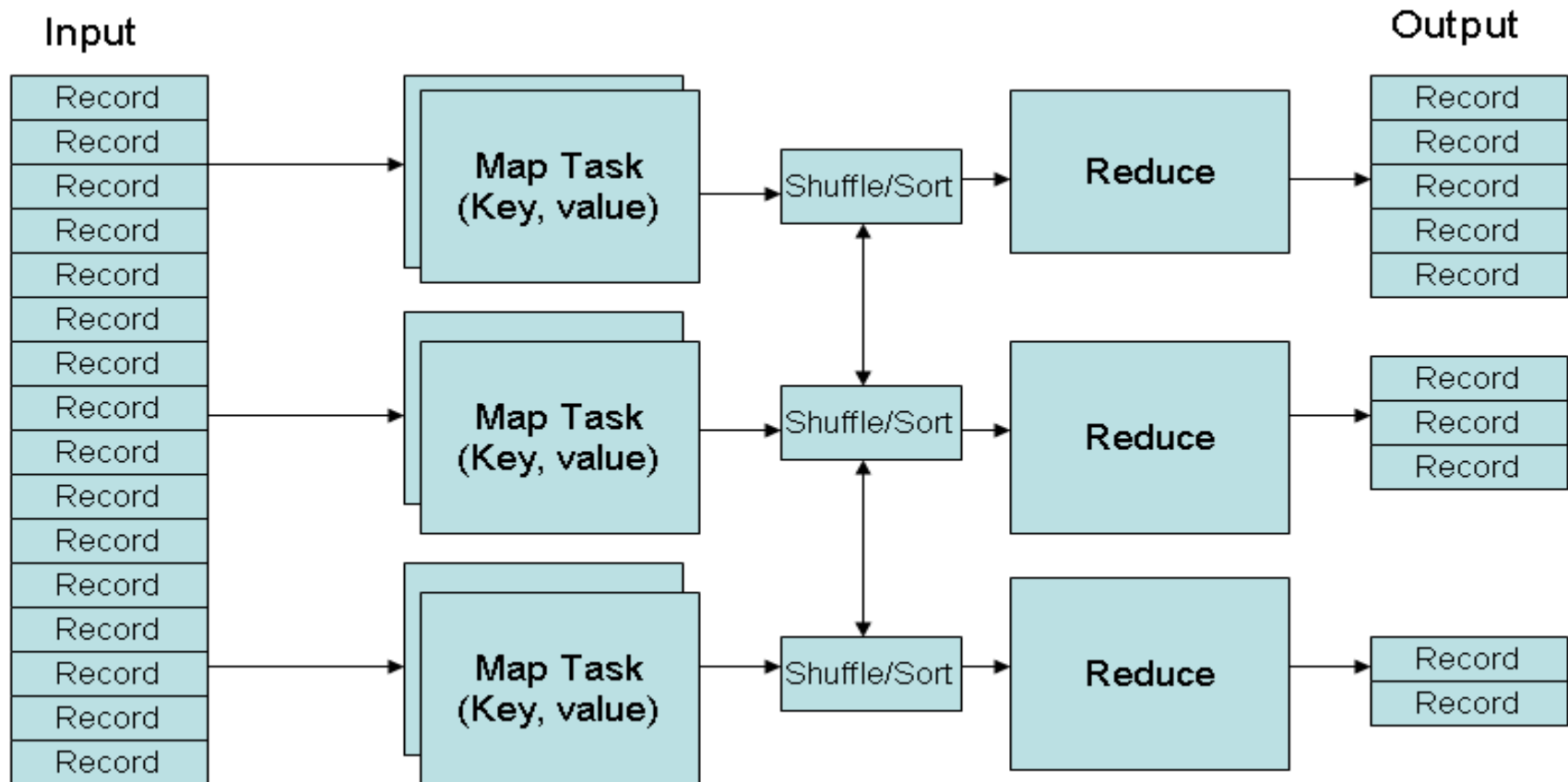
- Consume <key, <list of values>>
- Produce <key, value>

❖ **Shuffling and Sorting:**

- Hidden phase between mappers and reducers
- Groups all similar keys from all mappers, sorts and passes them to a certain reducer in the form of <key, <list of values>>

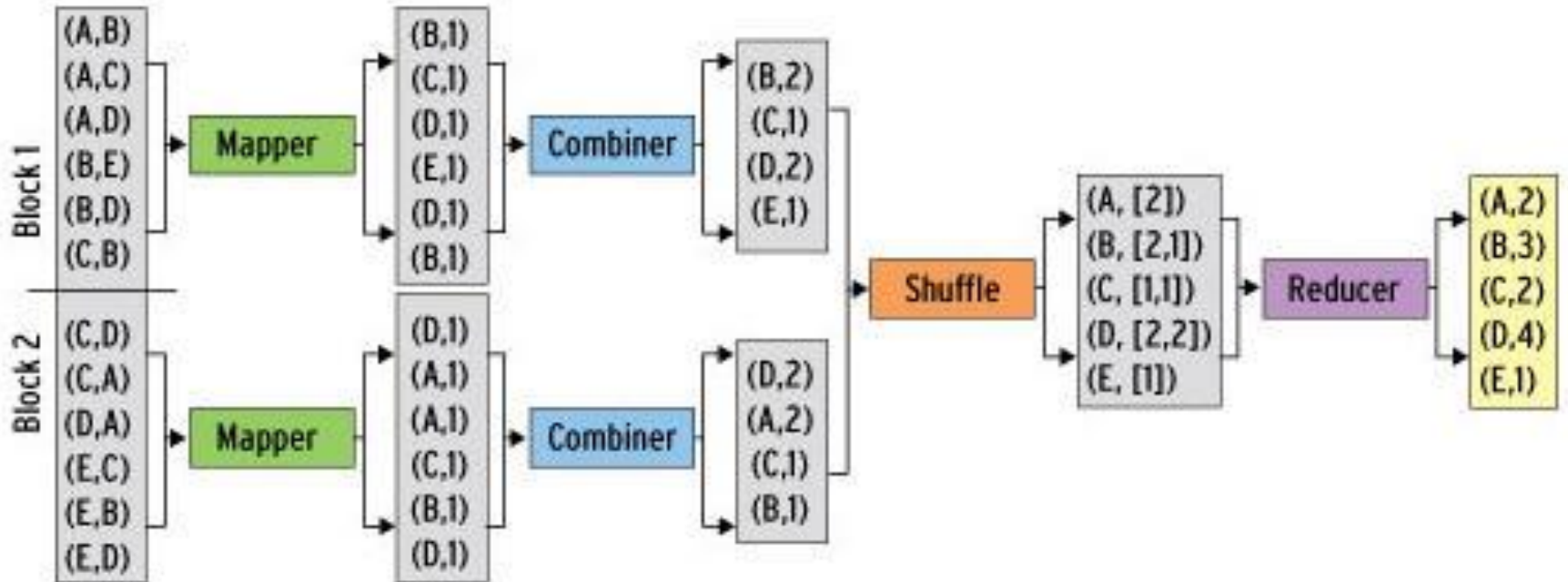
MapReduce Phases

☀ Deciding on what will be the key and what will be the value → developer's responsibility



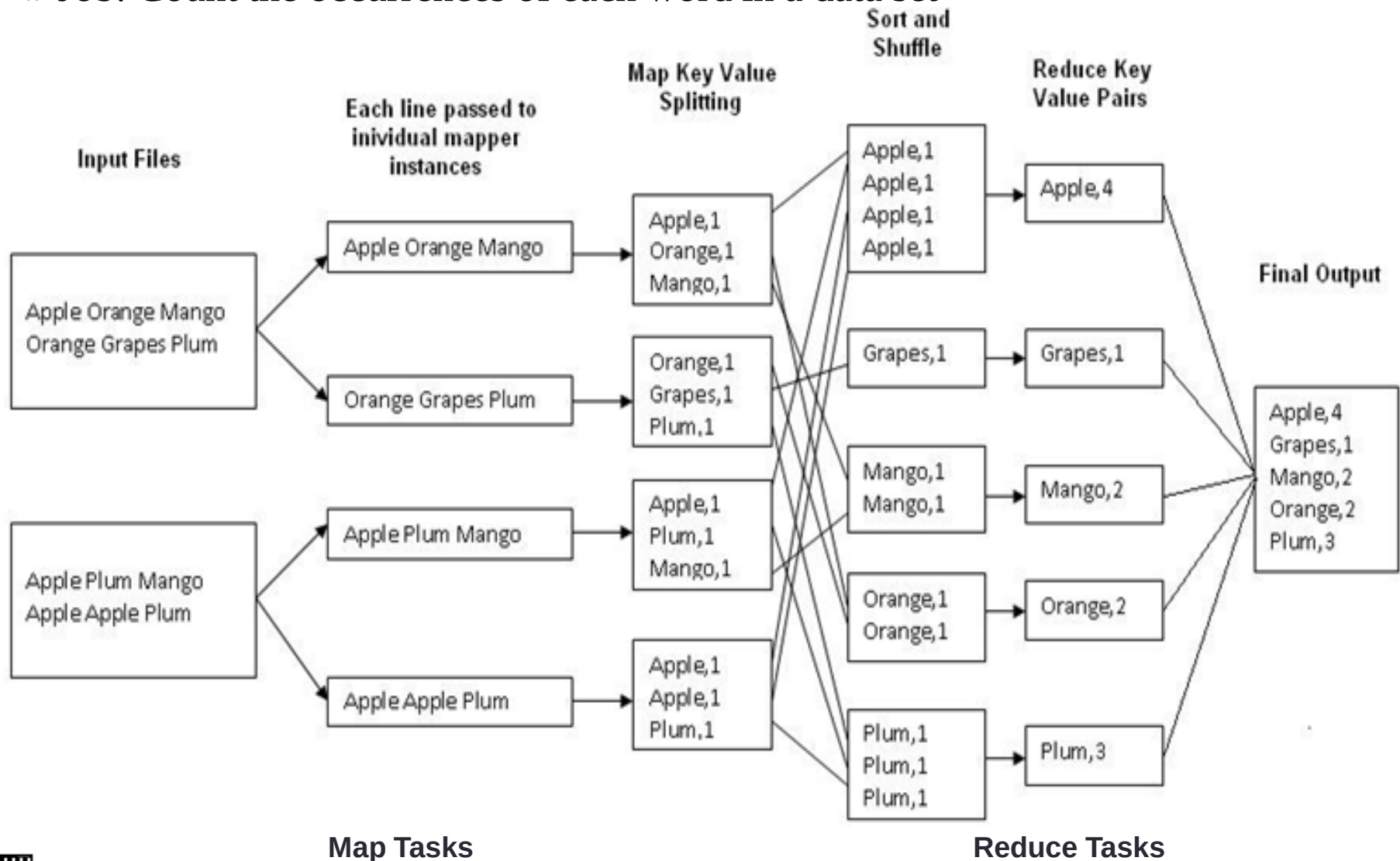
Refinement: Combiners

- ❖ Combiner combines values of all keys of a single mapper (single machine)
- ❖ Much less data needs to be copied and shuffled!



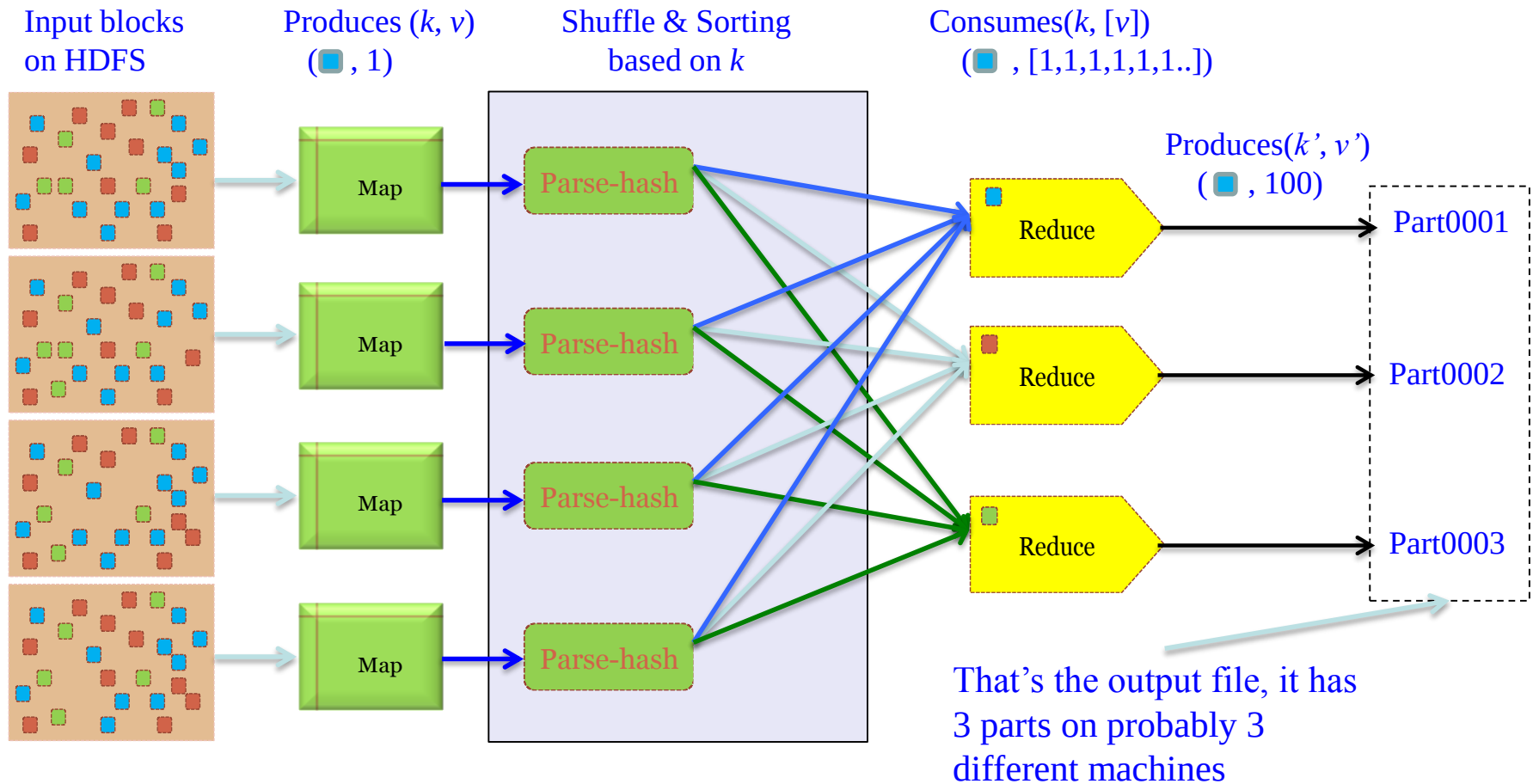
Example 1: Word Count

- Job: Count the occurrences of each word in a data set



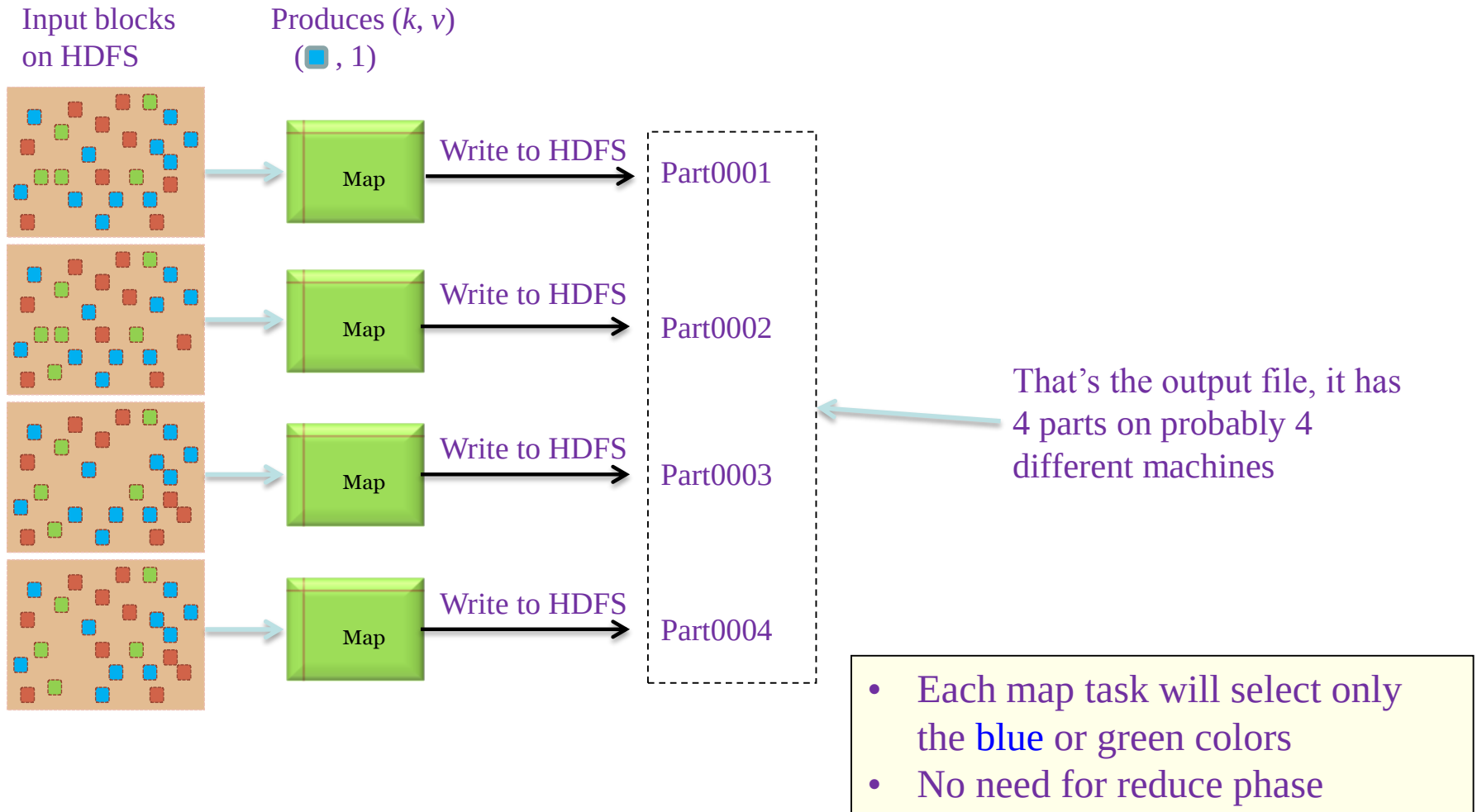
Example 2: Color Count

 **Job: Count the number of each color in a data set**



Example 3: Color Filter

Job: Select only the blue and the green colors



Other Hadoop Software Components

🔴 **Hadoop:** It is Apache's open source **software framework** for **storing, processing** and **analyzing** big data

🌱 Other software components that can run on top of or alongside Hadoop and have achieved top-level Apache project status include:

🌱 **Hive:** A data warehousing and SQL-like query language that presents data in the form of tables. Hive programming is similar to database programming.

🌱 **Ambari:** A web interface for managing, configuring and testing Hadoop services and components.

🌱 **Cassandra:** A distributed database system.

🌱 **Flume:** Software that collects, aggregates and moves large amounts of streaming data into HDFS.

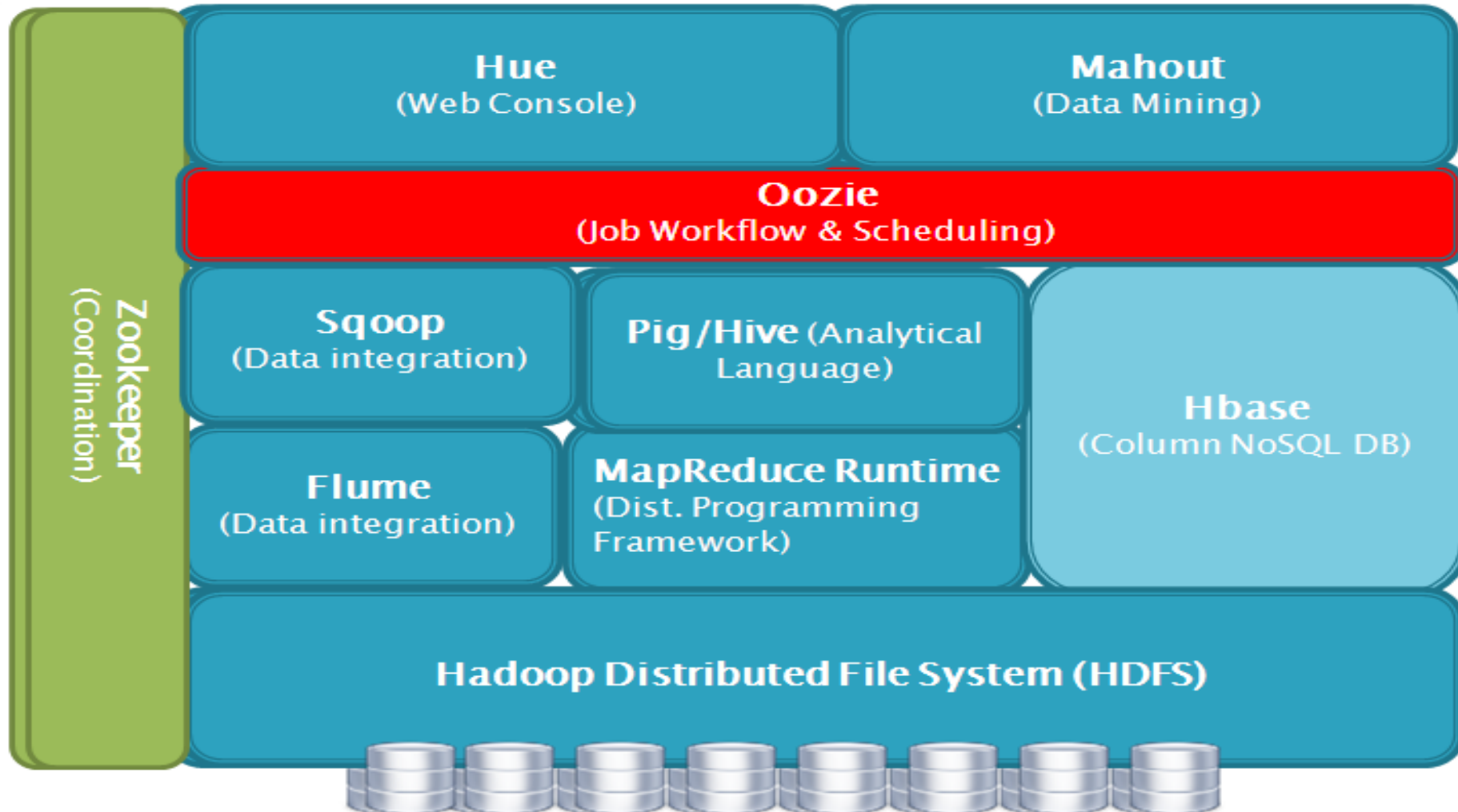
🌱 **Hbase:** A nonrelational, distributed database that runs on top of Hadoop. HBase tables can serve as input and output for MapReduce jobs.

Other Software Components

- ❖ **Hcatalog:** A table and storage management layer that helps users share and access data.
- ❖ **Oozie:** A Hadoop job scheduler.
- ❖ **Pig:** A platform for manipulating data stored in HDFS that includes a compiler for MapReduce programs and a high-level language called Pig Latin. It provides a way to perform data extractions, transformations and loading, and basic analysis without having to write MapReduce programs.
- ❖ **Solr:** A scalable search tool that includes indexing, reliability, central configuration, failover and recovery.
- ❖ **Spark:** An open-source cluster computing framework with in-memory analytics.
- ❖ **Sqoop:** A connection and transfer mechanism that moves data between Hadoop and relational databases.
- ❖ **Zookeeper:** An application that coordinates distributed processing.

Hadoop Ecosystem

It is Apache's open source **software framework** for **storing, processing** and **analyzing** big data.



Hadoop Ecosystem

 Hbase

 Hive

 Pig

 Flume

 Sqoop

 Oozie

 Hue

 Mahout

 Zookeeper

Hadoop Ecosystem

➤ **Hive**

- It is SQL-like interface to Hadoop.
- It is an abstraction on top of MapReduce. allows users to query data in the Hadoop Cluster without knowing java or MapReduce.
- Uses HiveQL language very similar to SQL.

➤ **Pig**

- Pig is an alternative abstraction on top of MapReduce.
- It uses a dataflow scripting language called as PigLatin.
- The Pig interpreter runs on the client machine.
- Takes the PigLatin script and turns it into a series of MapReduce jobs and submits those jobs to the cluster.

➤ Hbase

- HBase is "Hadoop Database".
- It is 'NoSQL' data store.
- It can store massive amount of data e.g. Gigabyte, terabyte or even pet bytes of data in a table.
- MapReduce is not designed for iterative processes.

➤ Flume

- It is distributed real time data collection service.
- It efficiently collects, aggregate and move large amounts of data.

➤ **Sqoop**

- It provides a method to import data from tables in relational database into HDFS.
- It supports easy parallel database import/export.
- User can insert data from RDBMS to HDFS , Export data from HDFS to back into RDBMS.

➤ **Oozie**

- Oozie is workflow management project.
- Oozie allows developers to create a workflow of MapReduce jobs including dependencies between jobs.
- The Oozie server submits the jobs to the server in the correct sequence.

Hadoop Ecosystem

➤ Hue

- An open source web interface that supports Apache Hadoop and its ecosystem licensed under the Apache v2 license
- Hue aggregates the most common Apache Hadoop components into a single interface and targets the user experience/UI Toolkit

➤ Mahout

- Machine learning tool
- Supports distributed & scalable machine learning algo on Hadoop Platform
- Helps in building intelligent applications easier and faster
- It can provide distributed data mining function combined with Hadoop

➤ ZooKeeper

- It is a centralized service for maintaining configuration information .
- It provides distributed synchronization.
- It contains a set of tools to build distributed applications that can safely handle partial failures.
- Zookeeper was designed to store coordination data, status information, configuration , location information about distributed application.

Hadoop Limitations

-  Security Concerns
-  Vulnerable by Nature
-  Not fit for small data
-  Potential Stability Issues
-  General limitations

Hadoop fs Shell Commands Examples

 `hadoop fs <args>`

 `hadoop fs -ls /user/hadoop/students`

 `hadoop fs -mkdir /user/hadoop/hadoopdemo`

 `hadoop fs -cat /user/hadoop/dir/products/products.dat`

 `hadoop fs -copyToLocal /user/hadoop/hadoopdemo/sales salesdemo`

 `hadoop fs -put localfile /user/hadoop/hadoopdemo`

 `hadoop fs -rm /user/hadoop/file`

 `hadoop fs -rmr /user/hadoop/dir`

- 🌸 Advertisement (Mining user behavior to generate recommendations)
- 🌸 Searches (group related documents)
- 🌸 Security (search for uncommon patterns)
- 🌸 Non-realtime large dataset computing:
 - 🌸 NY Times was dynamically generating PDFs of articles from 1851-1922
 - 🌸 Wanted to pre-generate & statically serve articles to improve performance
 - 🌸 Using Hadoop + MapReduce running on EC2 / S3, converted 4TB of TIFFs into 11 million PDF articles in 24 hrs

🌸 Hadoop is in use at most organizations that handle big data:

🌸 Yahoo!

🌸 Facebook

🌸 Amazon

🌸 Netflix

🌸 Etc...

🌸 Some examples of scale:

🌸 Yahoo!'s Search Webmap runs on 10,000 core Linux cluster and powers Yahoo! Web search

🌸 FB's Hadoop cluster hosts 100+ PB of data (July, 2012) & growing at ½ PB/day (Nov, 2012)

Course: Big Data Analytics

Module 1: Introduction to Big data

Module 2 : Introduction to Hadoop

Module 3 : NoSQL

Module 4: MapReduce and New S/w Stack

Module 5 : Finding similar Items

Module 6 : Mining Data Stream

Module 7 : Link Analysis

Module 8: Frequent item set

Module 9 : Clustering

Module 10 : Recommendation system

Module 11: Mining Social-Networks Graphs

References

✳ Text Books:

1. Anand Rajaraman and Jeff Ullman “**Mining of Massive Datasets**”, Cambridge University Press,
2. Alex Holmes “Hadoop in Practice”, Manning Press, DreamTech Press.
3. Dan McCreary and Ann Kelly “**Making Sense of NoSQL**” – **A guide for managers and the rest of us**, Manning Press.
4. “Big Data: The next frontier for innovation, competition, and productivity”. Rapporto McKinsey&Company, 2012.
5. “Challenges and Opportunities with Big Data”. A community white paper developed by leading researchers across the United States, 2012.
6. “Taming The Big Data Tidal Wave: Finding Opportunities in Huge Data Streams with Advanced Analytics”. Bill Franks, John Wiley & Sons, 2012.

✳ References:

1. Bill Franks , “**Taming The Big Data Tidal Wave: Finding Opportunities In Huge Data Streams With Advanced Analytics**”, Wiley
2. Chuck Lam, “**Hadoop in Action**”, Dreamtech Press
3. Judith Hurwitz, Alan Nugent, Dr. Fern Halper, Marcia Kaufman, “**Big Data for Dummies**”, Wiley India
4. Michael Minelli, Michele Chambers, Ambiga Dhiraj, “**Big Data Big Analytics: Emerging Business Intelligence And Analytic Trends For Today's Businesses**”, Wiley India
5. Phil Simon, “**Too Big To Ignore: The Business Case For Big Data**”, Wiley India
6. Paul Zikopoulos, Chris Eaton, “**Understanding Big Data: Analytics for Enterprise Class Hadoop and Streaming Data**’, McGraw Hill Education.
7. Boris Lublinsky, Kevin T. Smith, Alexey Yakubovich, “**Professional Hadoop Solutions**”, Wiley India.

Thank You.



Mining Data Streams (Part 1)

CS345a: Data Mining
Jure Leskovec and Anand Rajaraman
Stanford University

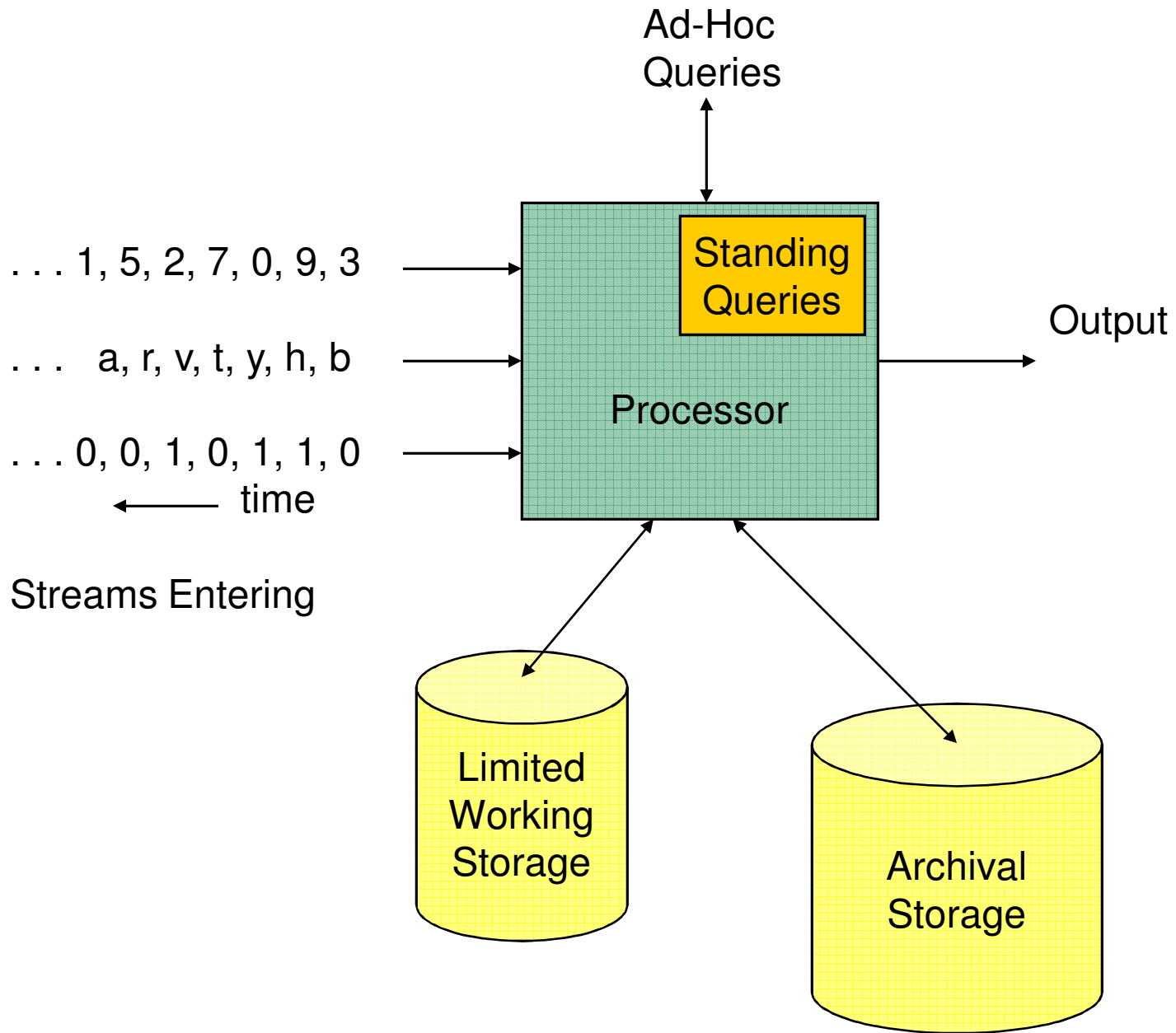


Data Streams

- In many data mining situations, we know the entire data set in advance
- Sometimes the input rate is controlled externally
 - Google queries
 - Twitter or Facebook status updates

The Stream Model

- Input tuples enter at a rapid rate, at one or more input ports.
- The system cannot store the entire stream accessibly.
- How do you make critical calculations about the stream using a limited amount of (secondary) memory?



Applications – (1)

- Mining query streams
 - Google wants to know what queries are more frequent today than yesterday
- Mining click streams
 - Yahoo wants to know which of its pages are getting an unusual number of hits in the past hour
- Mining social network news feeds
 - E.g., Look for trending topics on Twitter, Facebook

Applications – (2)

- Sensor Networks
 - Many sensors feeding into a central controller
- Telephone call records
 - Data feeds into customer bills as well as settlements between telephone companies
- IP packets monitored at a switch
 - Gather information for optimal routing
 - Detect denial-of-service attacks

Data Stream Problems

- Sampling data from a stream
- Filtering a data stream
- Queries over sliding windows
- Counting distinct elements
- Estimating moments
- Finding frequent elements
- Frequent itemsets

Sampling from a Data Stream

- Since we can't store the entire stream, one obvious approach is to store a sample
- Two different problems:
 - Sample a fixed proportion of elements in the stream (say 1 in 10)
 - Maintain a random sample of fixed size over a potentially infinite stream

Sampling a fixed proportion

- Scenario: search engine query stream
 - Tuples: (user, query, time)
 - Answer questions such as: how often did a user run the same query on two different days?
 - Have space to store $1/10^{\text{th}}$ of query stream
- Naïve solution
 - Generate a random integer in $[0..9]$ for each query
 - Store query if the integer is 0, otherwise discard

Problem with naïve approach

- Consider the question: What fraction of queries by an average user are duplicates?
- Suppose each user issues s queries once and d queries twice (total of $s+2d$ queries)
 - Correct answer: $d/(s+2d)$
 - Sample will contain $s/10$ of the singleton queries and $2d/10$ of the duplicate queries at least once
 - But only $d/100$ pairs of duplicates
 - So the sample-based answer is: $d/(10s+20d)$

Solution

- Pick $1/10^{\text{th}}$ of **users** and take all their searches in the sample
- Use a hash function that hashes the user name or user id uniformly into 10 buckets

Generalized Solution

- Stream of tuples with keys
 - Key is some subset of each tuple's components
 - E.g., tuple is (user, search, time); key is **user**
 - Choice of key depends on application
- To get a sample of size a/b
 - Hash each tuple's key uniformly into b buckets
 - Pick the tuple if its hash value is at most a

Maintaining a fixed-size sample

- Suppose we need to maintain a sample of size exactly s
 - E.g., main memory size constraint
- Don't know length of stream in advance
 - In fact, stream could be infinite
- Suppose at time t we have seen n items
 - Ensure each item is in sample with equal probability s/n

Solution

- Store all the first s elements of the stream
- Suppose we have seen $n-1$ elements, and now the n^{th} element arrives ($n > s$)
 - With probability s/n , pick the n^{th} element, else discard it
 - If we pick the n^{th} element, then it replaces one of the s elements in the sample, picked at random
- Claim: this algorithm maintains a sample with the desired property

Proof: By Induction

- Assume that after n elements, the sample contains each element seen so far with probability s/n
- When we see element $n+1$, it gets picked with probability $s/(n+1)$
- For elements already in the sample, probability of remaining in the sample is:

$$\left(1 - \frac{s}{n+1}\right) + \left(\frac{s}{n+1}\right)\left(\frac{s-1}{s}\right) = \frac{n}{n+1}$$

Sliding Windows

- A useful model of stream processing is that queries are about a *window* of length N – the N most recent elements received.
- **Interesting case:** N is so large it cannot be stored in memory, or even on disk.
 - Or, there are so many streams that windows for all cannot be stored.

q w e r t y u i o p **a s d f g h** j k l z x c v b n m

q w e r t y u i o p **a s d f g h** j k l z x c v b n m

q w e r t y u i o p a **s d f g h j** k l z x c v b n m

q w e r t y u i o p a s **d f g h j k** l z x c v b n m

← Past Future →

Counting Bits – (1)

- **Problem:** given a stream of 0's and 1's, be prepared to answer queries of the form “how many 1's in the last k bits?” where $k \leq N$.
- **Obvious solution:** store the most recent N bits.
 - When new bit comes in, discard the $N + 1^{\text{st}}$ bit.

Counting Bits – (2)

- You can't get an exact answer without storing the entire window.
- **Real Problem:** what if we cannot afford to store N bits?
 - E.g., we're processing 1 billion streams and $N = 1$ billion
- But we're happy with an approximate answer.

DGIM* Method

- Store $O(\log^2 N)$ bits per stream.
- Gives approximate answer, never off by more than 50%.
 - Error factor can be reduced to any fraction > 0 , with more complicated algorithm and proportionally more stored bits.

*Datar, Gionis, Indyk, and Motwani

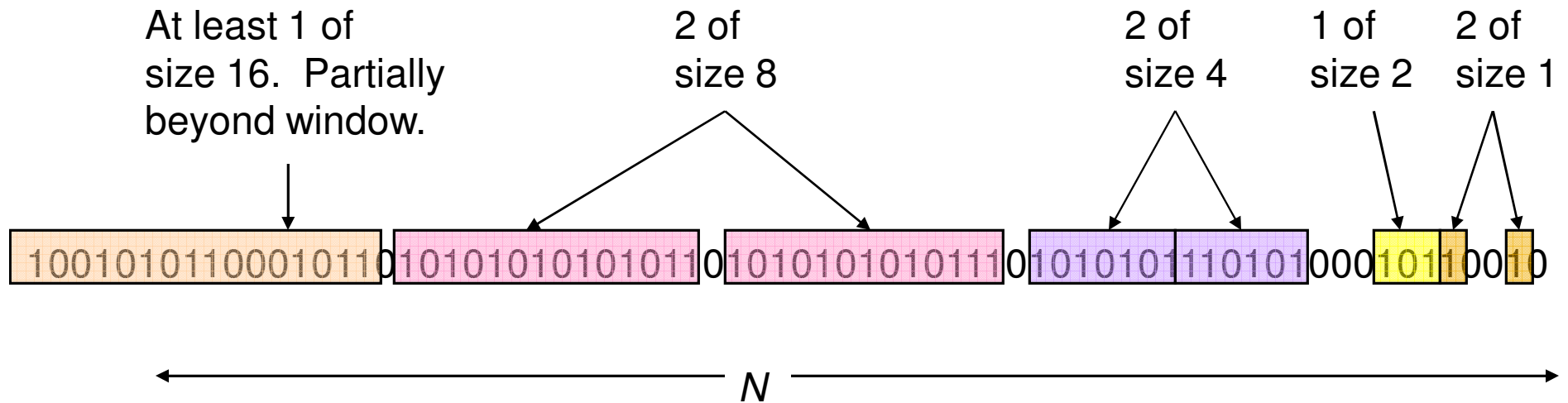
Something That Doesn't (Quite) Work

- Summarize exponentially increasing regions of the stream, looking backward.
- Drop small regions if they begin at the same point as a larger region.

Key Idea

- Summarize blocks of stream with specific numbers of 1's.
- Block *sizes* (number of 1's) increase exponentially as we go back in time

Example: Bucketized Stream



Timestamps

- Each bit in the stream has a *timestamp*, starting 1, 2, ...
- Record timestamps modulo N (the window size), so we can represent any *relevant* timestamp in $O(\log_2 N)$ bits.

Buckets

- A *bucket* in the DGIM method is a record consisting of:
 1. The timestamp of its end [$O(\log N)$ bits].
 2. The number of 1's between its beginning and end [$O(\log \log N)$ bits].
- **Constraint on buckets:** number of 1's must be a power of 2.
 - That explains the $\log \log N$ in (2).

Representing a Stream by Buckets

- Either one or two buckets with the same power-of-2 number of 1's.
- Buckets do not overlap in timestamps.
- Buckets are sorted by size.
 - Earlier buckets are not smaller than later buckets.
- Buckets disappear when their end-time is $> N$ time units in the past.

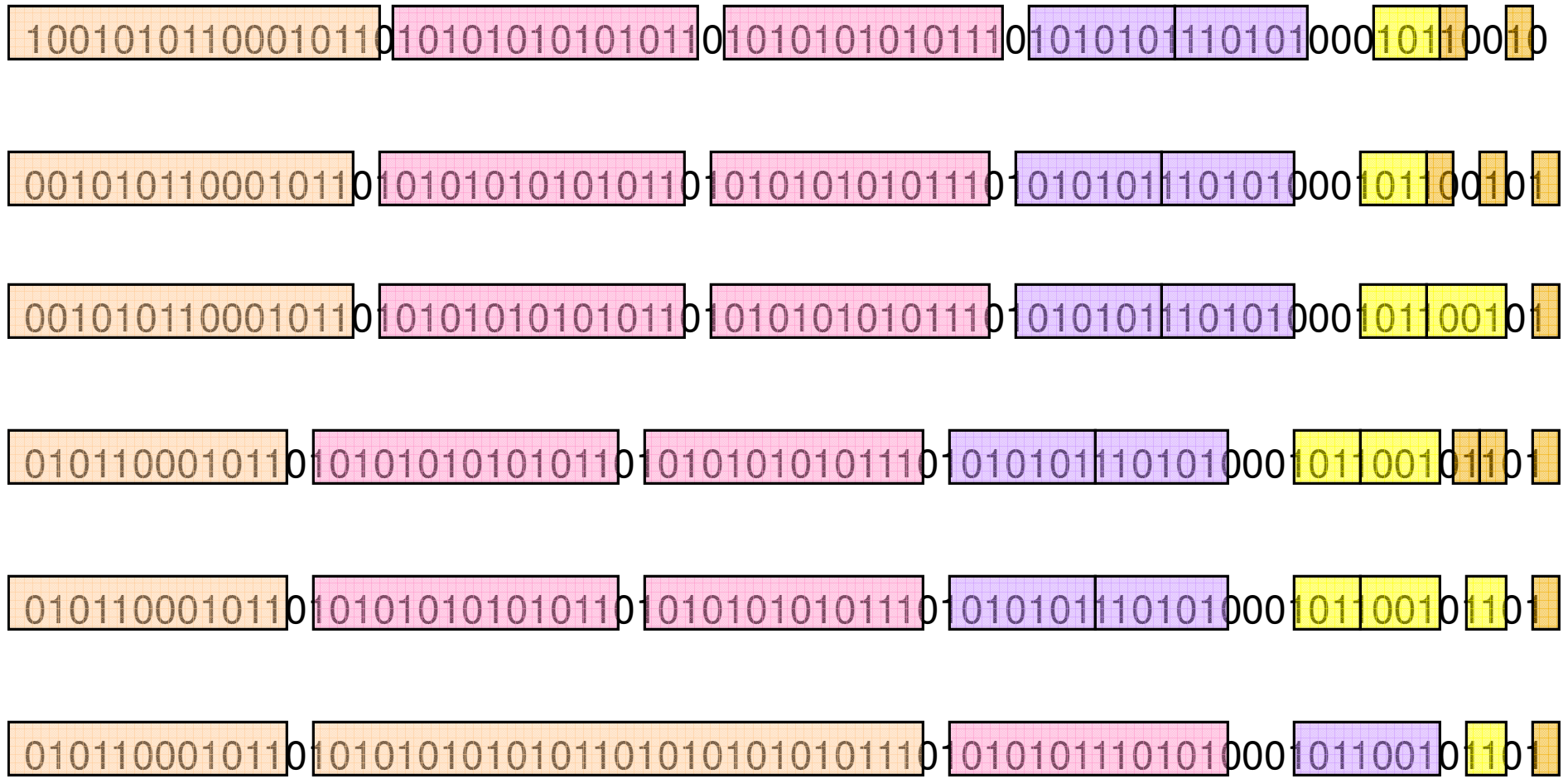
Updating Buckets – (1)

- When a new bit comes in, drop the last (oldest) bucket if its end-time is prior to N time units before the current time.
- If the current bit is 0, no other changes are needed.

Updating Buckets – (2)

- If the current bit is 1:
 1. Create a new bucket of size 1, for just this bit.
 - ◆ End timestamp = current time.
 2. If there are now three buckets of size 1, combine the oldest two into a bucket of size 2.
 3. If there are now three buckets of size 2, combine the oldest two into a bucket of size 4.
 4. And so on ...

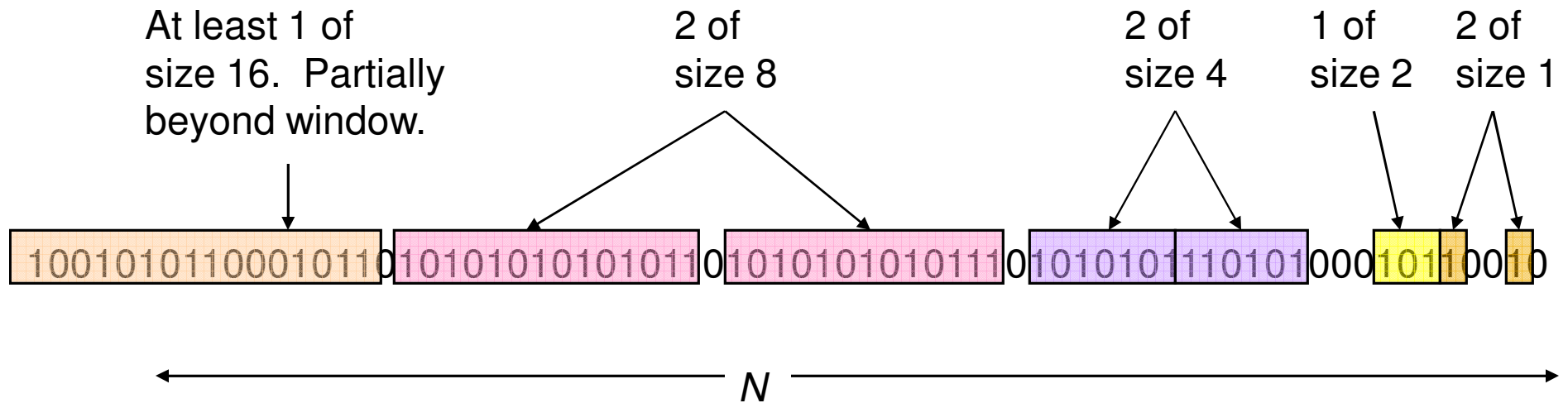
Example



Querying

- To estimate the number of 1's in the most recent N bits:
 1. Sum the sizes of all buckets but the last.
 2. Add half the size of the last bucket.
- **Remember:** we don't know how many 1's of the last bucket are still within the window.

Example: Bucketized Stream



Error Bound

- Suppose the last bucket has size 2^k .
- Then by assuming 2^{k-1} of its 1's are still within the window, we make an error of at most 2^{k-1} .
- Since there is at least one bucket of each of the sizes less than 2^k , the true sum is at least $1 + 2 + \dots + 2^{k-1} = 2^k - 1$.
- Thus, error at most 50%.

Extensions

- Can we use the same trick to answer queries “How many 1’s in the last k ?” where $k < N$?
- Can we handle the case where the stream is not bits, but integers, and we want the sum of the last k ?

Reducing the Error

- Instead of maintaining 1 or 2 of each size bucket, we allow either $r-1$ or r for $r > 2$
 - Except for the largest size buckets; we can have any number between 1 and r of those
- Error is at most by $1/(r-1)$
- By picking r appropriately, we can tradeoff between number of bits and error